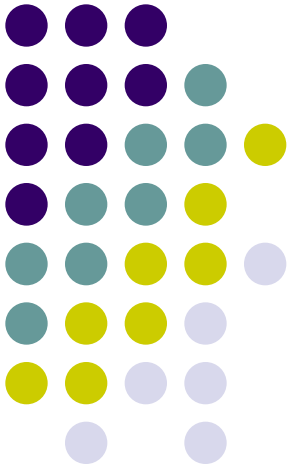
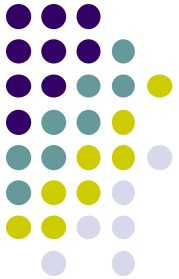
Natural Language Processing

CPSC 488

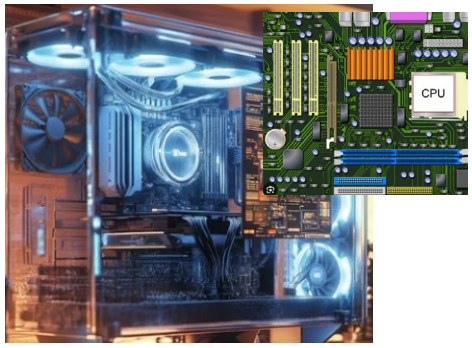
High-Performance Computing (HPC) and Deep Learning Frameworks



Lecture Overview



- **Basics of High-Performance Computing** (HPC)
  - Computer organization and parallel processing
  - Supercomputer and GPU accelerator
  - HPC frameworks and programming models
- **GPUs**, **CUDA**, and **HPC programming models**
- **The Nautilus GPU cluster** by National Research Platform
  - The Nautilus architecture
  - Deploying containerized applications
  - Kubernetes for container orchestration
- **Machine learning packages and deep learning frameworks**



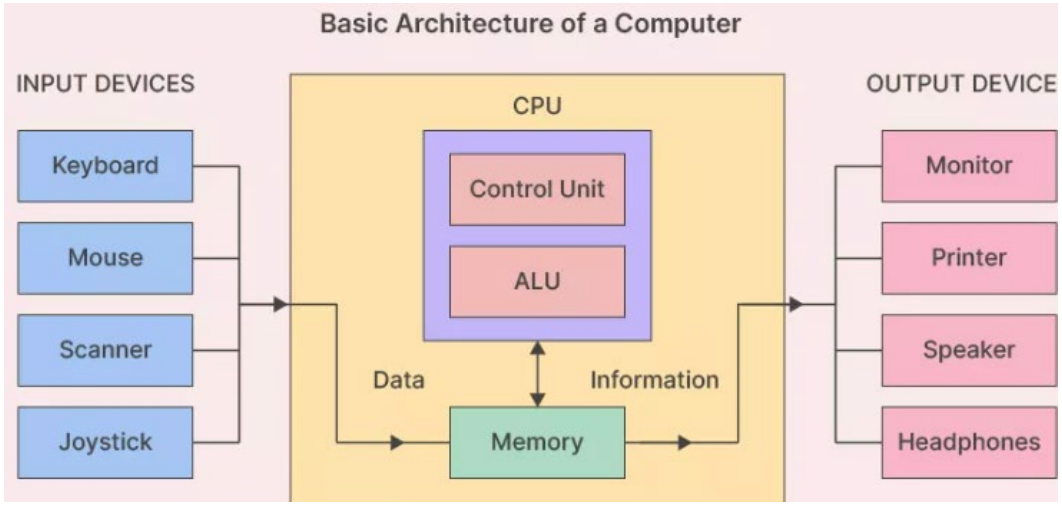
Basics of Computer Organization

GPU cards

Japan's Fugaku supercomputer

Computer Organization

- **Von Neuman architecture**



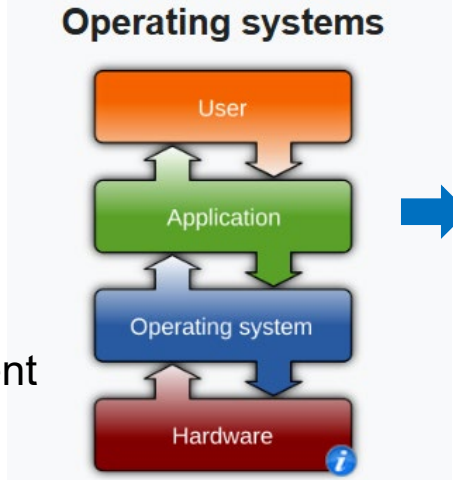
CPUs

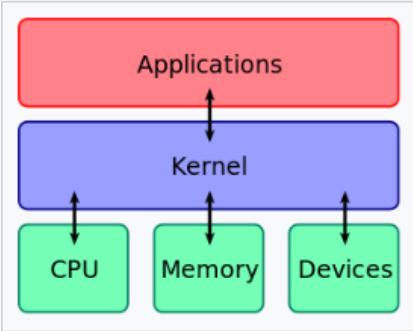
  - The **Central Processing Unit** (**CPU**) consists of:
    - Control Unit (CU), Arithmetic Logic Unit (ALU), and Memory within CPU (registers for high-speed storage, cache for high-speed storage)
  - **Random Access Memory** (RAM) and external devices to the CPU.
- **CPU** vs **Processor**
  - The processor commonly can refer to the CPU but can also refer to the other processing units like GPU, APU, etc.

Operating System (OS) and Kernel

The OS manages and provides necessary resources:

- CPU scheduling
- Memory management
- I/O operations



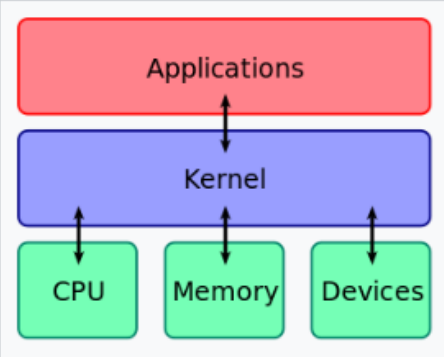


An oversimplification of how a kernel connects application software to the hardware of a computer

The **kernel** is the core of OS that manages the operations of computers and hardware.

- **How is a program loaded and executed?**
  1. The OS loads the program file (binary after compiling) and all the external library dependencies (after linking) into memory (code, data, stack, and heap sections).
  2. The OS creates a process for the program with a unique ID (PID).
  3. The CPU executes the program.

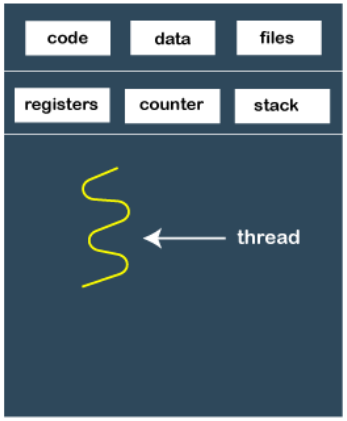
The Program Execution by the CPU



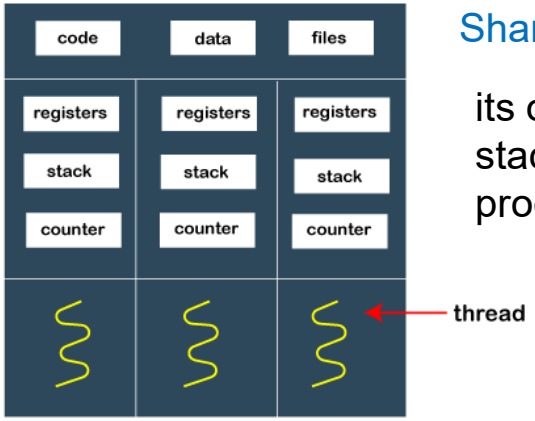
An oversimplification of how a kernel connects application software to the hardware of a computer

- **The CPU executes the program by:**
  - Fetching instructions pointed by the Program Counter (PC)
  - Decoding the instructions by the control unit (CU) of the CPU to determine the operation to perform
  - Executing the instruction (arithmetic/logic, memory read/write, or I/O operations) in the Arithmetic Logic Unit (ALU)
  - Updating PC
  - Interrupt handling (e.g., I/O requests, system calls)
  - Context switching for **multitasking**

Multitasking: Process and Thread



Single-threaded process



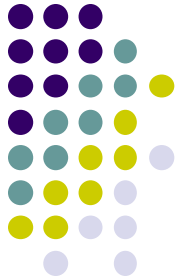
Multi-threaded process

Shared

its own registers, stack, and program counter

- **Process**
  - An independent program in execution, with its own memory space and resources.
  - For independent tasks running on different cores.
  - Inter-Process Communication (IPC) needed between processes
- **Thread**
  - The smallest unit of execution within a process, also called lightweight process for **concurrent tasks** in one program.
  - Threads share the process's memory and resources (faster) but can execute independently.
  - So, multi-threading will be faster than single-threading in multicore computers or for I/O bound tasks.

Examples: Multi-Process and -Thread



Multi-processing

```python
from multiprocessing import Process, current_process
import os

def worker(task_id):
    print(f"Process-{task_id} starting (PID: {os.getpid()})")
    result = sum(range(10_000_000))  # Simulate a CPU-intensive computation
    print(f"Process-{task_id} finished (PID: {os.getpid()}), Result: {result}")

if __name__ == "__main__":
    processes = []

    # Create and start processes
    for i in range(4):  # Create 4 processes
        process = Process(target=worker, args=(i,))
        processes.append(process)
        process.start()

    # Wait for all processes to finish
    for process in processes:
        process.join()

    print("All processes have completed.")
```

Heavier, for isolated tasks in microservices

Both process and thread can run on multicores.

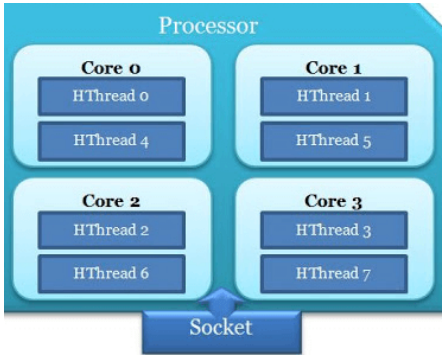
Multi-threading

```python
import threading
import time

def worker(task_id):
    print(f"Thread-{task_id} starting")
    time.sleep(2)  # Simulates an I/O operation
    print(f"Thread-{task_id} finished")

if __name__ == "__main__":
    threads = []

    # Create and start threads
    for i in range(5):  # Create 5 threads
        thread = threading.Thread(target=worker, args=(i,))
        threads.append(thread)
        thread.start()

    # Wait for all threads to finish
    for thread in threads:
        thread.join()

    print("All threads have completed.")
```

Lighter, for shared-state tasks

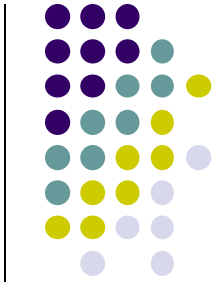
Multitasking: Concurrent vs Parallel

- **Concurrent processing vs Parallel processing**
  - Concurrent processing
    - A single-core CPU executing multiple tasks seemingly simultaneously running multiple threads or processes via time-slicing
  - Parallel processing
    - Multi-core CPUs executing multiple tasks simultaneously



- **CPU vs Core vs Logical processor**
  - A **core** is an individual processing unit **within** a CPU.
    - Each core has its own ALU, CU, and registers.
    - Multi-cores within a CPU enable it to perform parallel processing
    - Modern PC have multiple cores. Supercomputers have thousands of CPUs.
  - A **logical processor** is a **virtual** processor core.
    - A single physical core is divided into multiple logical units, created through Simultaneous Multithreading (SMT)
    - Appears as two or more processors **to the OS** for better utilizing its resources and improving performance in multi-threaded tasks, e.g., Intel's 4 cores with Hyper-Threading (a form of SMT) seen as having 8 logical processors to the OS.

Various Types of Processors

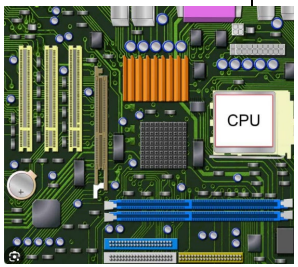


- **Central Processing Unit (CPU)**
  - For general purpose computing. Usually 4-6 cores. Intel, AMD
- **Graphics Processing Unit (GPU)**
  - For massive parallel processing of images, videos, other types of data
  - Thousands of smaller cores. **NVIDIA**, AMD, Intel
    - NVIDIA GeForce RTX 4090 has over 16k cores.
- **Tensor Processing Unit (TPU)**
  - Specialized processing for deep learning developed by Google
- **Neural Processing Unit (NPU)**
  - Specialized processing for neural networks, developed by NVIDIA, Apple, Google, Samsung, Qualcomm, Huawei, Intel
- **Accelerated Processing Unit (APU)**
  - Processor by **AMD** that integrates a CPU and GPU onto a single chip.
  - Cost-effective, power efficiency, compact
- **Quantum Processing Unit (QPU)**
  - Many companies including Google, IBM, Microsoft, etc.

Various Types of Computing

Laptop

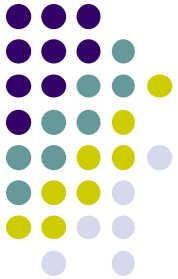
Desktop or workstation



Servers in a data center

Japan's Fugaku supercomputer

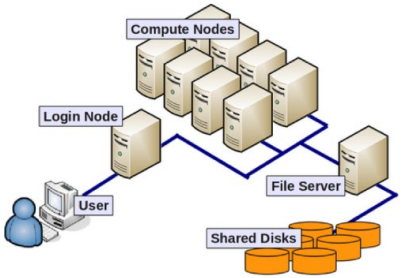
High-Performance Computing (HPC)



- **What is HPC?**
  - The use of advanced computational systems and techniques with huge computational power, memory, and data processing
    - Typically leveraging supercomputers, clusters, distributed systems and hardware.
- **Related computing technologies**
  - Parallel computing by supercomputers
    - Extremely fast computers with thousands of CPUs and cores, specially designed for large-scale parallel processing
  - Distributed computing, grid computing, clustered computing, cloud computing (in the order of evolution)
- **Types of HPC platforms**
  - Supercomputers: Summit, Sierra, Fugaku, etc.
  - Distributed and clustered computers: Nautilus, TACC, HP, Dell, etc.
  - Cloud: Amazon, Google, Microsoft
  - Computers with accelerators: PCs, workstations, or servers with GPUs

Types of HPC Platforms

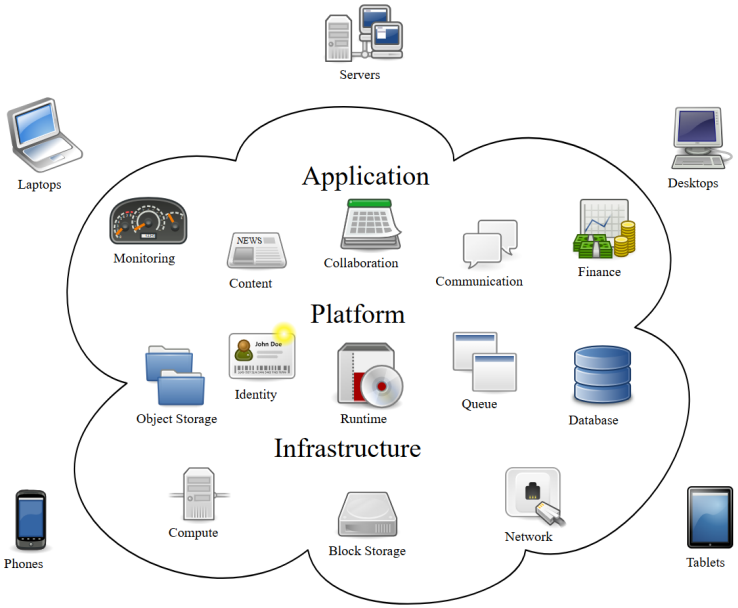
Clustered



Distributed

Servers in a data center

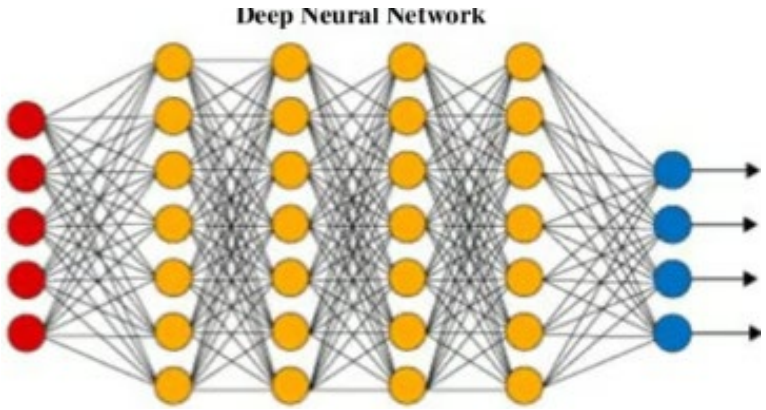
Cloud computing



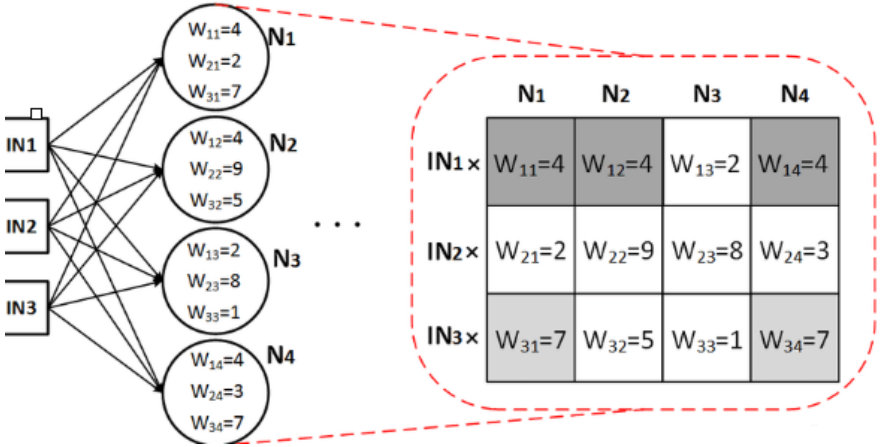
Supercomputer

Why HPC in AI?

- **Massive parallel computation required in most AI problems**



Weights learning in neural networks
The Transformer used for ChatGPT-3 has the total **number of connections** (parameters) 175B with 96 layers.



At each node, **weighed-sum** $\Sigma = \sum w_j x_{ij}$ is calculated and compute the output $\hat{y} = \phi(\Sigma)$.

Common Computation Types in AI

Vector multiplication or dot product

$$[1 \quad 2 \quad 3] \times \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix} = [1 \times 2 + 2 \times 1 + 3 \times 3] = 13$$

$$[4 \quad 5 \quad 6] \times \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix} = [4 \times 2 + 5 \times 1 + 6 \times 3] = 31$$

$$[7 \quad 8 \quad 9] \times \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix} = [7 \times 2 + 8 \times 1 + 9 \times 3] = 49$$

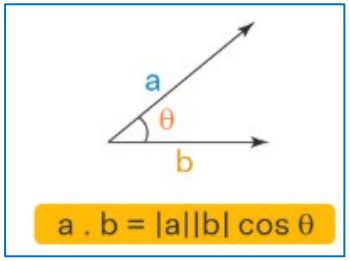
Matrix multiplication

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 10 & 11 \\ 20 & 21 \\ 30 & 31 \end{bmatrix}$$

$$= \begin{bmatrix} 1x10 + 2x20 + 3x30 & 1x11 + 2x21 + 3x31 \\ 4x10 + 5x20 + 6x30 & 4x11 + 5x21 + 6x31 \end{bmatrix}$$

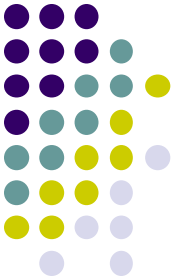
$$= \begin{bmatrix} 10+40+90 & 11+42+93 \\ 40+100+180 & 44+105+186 \end{bmatrix} = \begin{bmatrix} 140 & 146 \\ 320 & 335 \end{bmatrix}$$

- **Vector multiplication**
  - Dot product
    - The product of the magnitudes of the two vectors
  - Weighed-sum
    - $\Sigma = \sum w_j x_{ij}$ is calculated and compute the output $\hat{y} = \phi(\Sigma)$.
- **Vector matrix** or **matrix matrix multiplication**
  - Input data set (matrix) x parameters (vector)



dot product

Vector and Matrix Operations



Vector multiplication or dot product

$$[1 \quad 2 \quad 3] \times \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix} = [1 \times 2 + 2 \times 1 + 3 \times 3] = 13$$

$$[4 \quad 5 \quad 6] \times \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix} = [4 \times 2 + 5 \times 1 + 6 \times 3] = 31$$

$$[7 \quad 8 \quad 9] \times \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix} = [7 \times 2 + 8 \times 1 + 9 \times 3] = 49$$

Matrix multiplication

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 10 & 11 \\ 20 & 21 \\ 30 & 31 \end{bmatrix}$$

$$= \begin{bmatrix} 1x10 + 2x20 + 3x30 & 1x11 + 2x21 + 3x31 \\ 4x10 + 5x20 + 6x30 & 4x11 + 5x21 + 6x31 \end{bmatrix}$$

$$= \begin{bmatrix} 10+40+90 & 11+42+93 \\ 40+100+180 & 44+105+186 \end{bmatrix} = \begin{bmatrix} 140 & 146 \\ 320 & 335 \end{bmatrix}$$

Translation matrix

$$\begin{bmatrix} X_{new} \\ Y_{new} \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & T_x \\ 0 & 1 & T_y \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{old} \\ Y_{old} \\ 1 \end{bmatrix}$$

Rotation matrix

$$\begin{bmatrix} X_{new} \\ Y_{new} \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{old} \\ Y_{old} \\ 1 \end{bmatrix}$$

Matrix Vector Multiplication

- For **regression modeling**
  - Learning parameters w to find a function

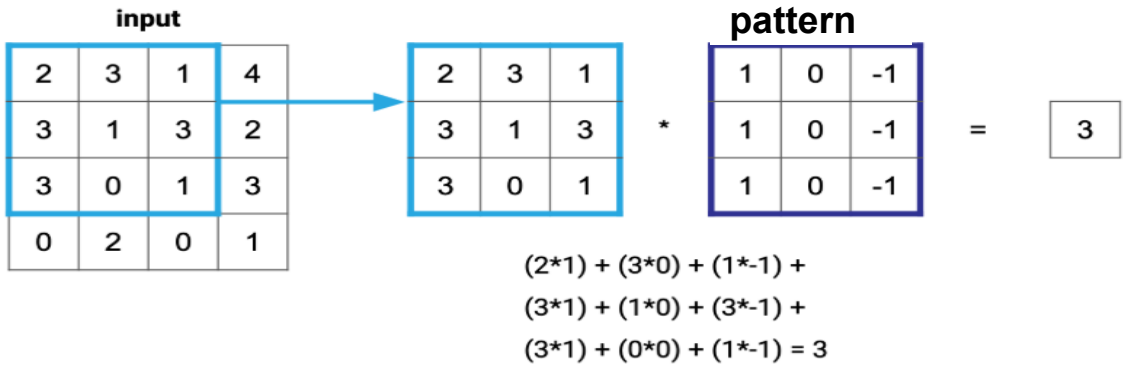
Price (K$) (Y)	*Size (X_1)*	*Bedroom (X_2)*	*Bathroom (X_3)*	*Built year (X_4)*
375 (y_1)	1024 (x_{11})	3 (x_{12})	2 (x_{13})	1978 (x_{14})
425 (y_2)	1329 (x_{21})	3 (x_{22})	5 (x_{23})	1992 (x_{24})
…	…	…	…	…
465 (y_N)	1893 (x_{N1})	4 (x_{N2})	4 (x_{N3})	1980 (x_{N4})

$$
\begin{aligned}
y_1 &= w_0 + w_1 x_{11} + \ldots + w_k x_{1k} \\
y_2 &= w_0 + w_1 x_{21} + \ldots + w_k x_{2k} \\
\ldots &\quad \ldots \quad \ldots \quad \ldots \quad \ldots \\
y_N &= w_0 + w_1 x_{N1} + \ldots + w_k x_{Nk}
\end{aligned}
$$

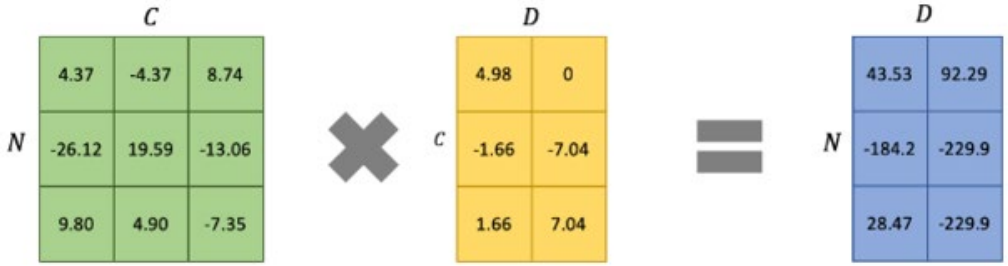
$$
\underbrace{\begin{bmatrix} y_1 \\ \ldots \\ y_N \end{bmatrix}}_{y} = \underbrace{\begin{bmatrix} 1 & x_{11} & \ldots & x_{1k} \\ \ldots & \ldots & \ldots & \ldots \\ 1 & x_{N1} & \ldots & x_{Nk} \end{bmatrix}}_{X} \underbrace{\begin{bmatrix} w_0 \\ \ldots \\ w_k \end{bmatrix}}_{w} \Rightarrow y = Xw \text{ or } y = w^T X
$$

Matrix Vector Multiplication

- For **image classification** or **object detection**
  - Finding patterns in images with convolution operation



- For **data compression/encoding**
  - Commonly used in language modeling



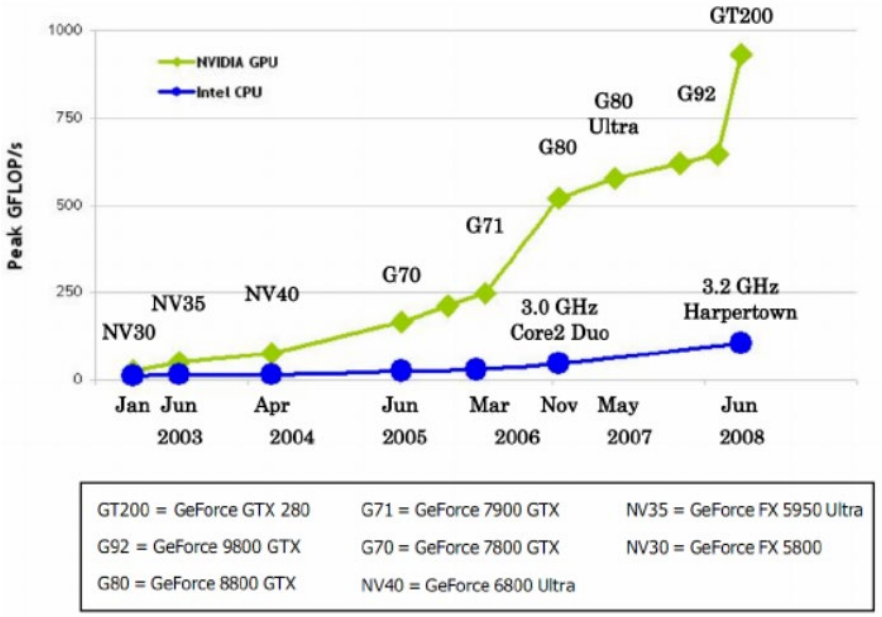
Measuring Computation Performance

Matrix multiplication

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 10 & 11 \\ 20 & 21 \\ 30 & 31 \end{bmatrix}$$

$$= \begin{bmatrix} 1x10 + 2x20 + 3x30 & 1x11 + 2x21 + 3x31 \\ 4x10 + 5x20 + 6x30 & 4x11 + 5x21 + 6x31 \end{bmatrix}$$

$$= \begin{bmatrix} 10+40+90 & 11+42+93 \\ 40+100+180 & 44+105+186 \end{bmatrix} = \begin{bmatrix} 140 & 146 \\ 320 & 335 \end{bmatrix}$$



- **Measuring computation speed**
  - **Floating-Point Operations Per Second** (**FLOPS**)
    - 1 petaflops: 1,000 teraflops or 10^{15} flops, 1 teraflops: 1,000 gigaflops
  - Theoretical peak FLOPS
    - The maximum number of floating-point operations per second based on its hard specifications
    - Peak FLOPS = # of cores x clock speed x FLOPs per cycle (per core)

HPC with Supercomputers

- **Characteristics of supercomputers** (compared to **other types of HPC**)
  - Centralized system based on integrated architecture and fast interconnection
  - Large number of CPUs and/or GPUs for extreme performance in general computing purpose (mathematical operation, loops, data access, communication, etc.)
  - Custom cooling and power system
  - High cost (CPUs) and less scalable
- **Supercomputers**
  - Supercomputers, early 2000s
    - IBM ASCI White (7 teraflops using **8,192 CPUs**, **375 MHz** per processor, 2000, fastest!),
    - IBM Blue Gene/L (136 teraflops, **2005**), **IBM Roadrunner** (1 petaflops, 2008)
  - US: El Capitan (1,742 petaflops with **44,544 AMD APUs**, each **APU** has 24 **CPU** cores and 4 **GPU** cores, total **1,068,096 cores**), Frontier (1,102 petaflops), Aurora (1000+ petaflops, **Intel** CPUs and GPUs), IBM Summit (148 petaflops, 2,416 **IBM** CPUs, 2,416 **NVIDIA** GPUs), Sierra (94 petaflops), etc.
  - Japan: Fugaku (442 petaflops with 158,976 ARM-based A64FX processors, **7,627,008** cores)
  - Finland, Europe: LUMI (309 petaflops, AMD CPUs, NVIDIA GPUs, 262,144 cores)
  - China: Sunway TaihuLight (94 petaflops, 40,960 SW26010 processors, **10,665,600** cores)

HPC with Computing Accelerators

- **Computers with accelerators**
  - Computers with accelerators such as GPU, TPU, FPGA, custom AI-chips
- **GPUs are accelerators.**
  - Specially designed for massive parallelism and (own) fast memory to support compute-intensive applications
  - Separate card **connected** to **CPU** via PCI-E bus
  - Accelerated floating workloads:
    - Peak Floating Point (FP) performance: >10x vs CPU
    - Memory bandwidth >20x vs CPU
- **Why GPUs for AI**
  - NVIDIA A100 GPU (**2020**): **312 teraflops** with 6912 cores
  - IBM Blue Gene/L supercomputer (2005): 136 teraflops with 4096 cores from 32 nodes
- **Applications**
  - Image and video processing, gaming
  - Deep learning, cryptocurrency mining, computational biology

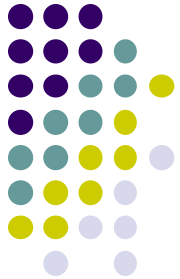
Computational Power Needed for AI

- **Training AI models requires lots of computations.**
  - For image classification, 650k neurons, 630 millions connections, and 4096-dimensions for the final feature layer, run stochastic gradient descent on two NVIDIA GPUs for a week.

NVIDIA GPU is good at computing vector or matrix operations in parallel.

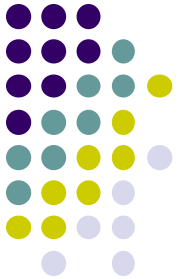
GPU clusters in a data center

Pros and Cons of Accelerators



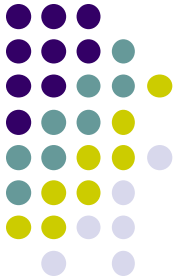
- **Benefits comparing with supercomputers**
  - Enhanced performance and better energy efficiency
  - Scalable, cost effective, specialized optimization and broader applications
  - Ideal use cases:
    - Single Instruction Multiple Data (SIMD)
    - Small, simple, and many independent operations
- **Challenges**
  - Programming complexity, portability and integration
  - Can be still high cost: accelerators and supporting infrastructure
  - Cases not recommended (cases where supercomputers are better.)
    - Multiple Instruction Multiple Data (MIMD): need more general-purpose CPUs
    - Large and complex tasks
    - Data intensive and iterative work, requiring frequent data transfer between CPU and GPU memories and process synchronization
- **Supercomputing systems with accelerators**
  - Frontier, Summit, Aurora, Fugaku, etc.

Software Frameworks for HPC

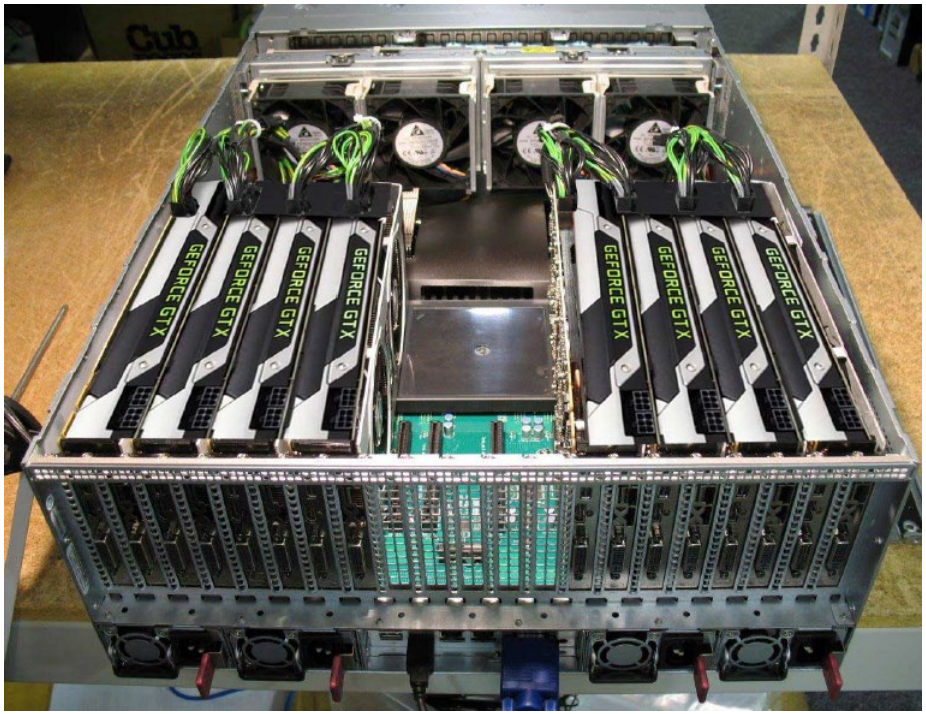


- **Parallel programming models for supercomputers**
  - Inter-Process Communication (IPC)
    - Message passing to support communication between processes running on different CPUs, which may also be across different nodes
    - Share-memory with a smaller number of processors
  - Programming models
    - Message passing model: Message Passing Interface (MPI)
    - Shared-memory model: Open Multi-Processing (OpenMP)
    - Hybrid: MPI + OpenMP
  - Programming languages:
    - C/C++
    - OpenCL
    - Fortran, Python, Julia, etc.
- **Many mathematical and scientific libraries**
  - Intel Math Kernel Library (MKL), Fast Fourier Transform in the West (FFTW), etc.

Software Frameworks for HPC

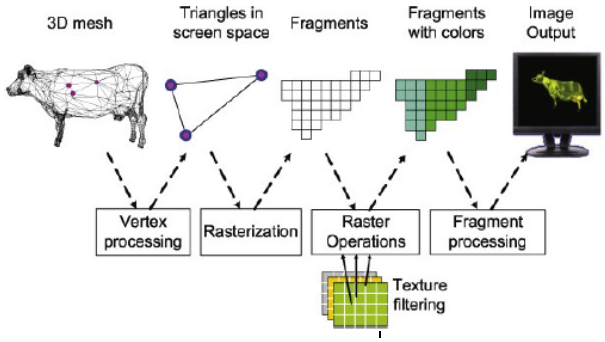


- **Distributed computing frameworks**
  - Apache Hadoop and Spark to utilize clustered computers or cloud
- **Accelerated programming frameworks with GPUs**
  - Compute Unified Device Architecture (**CUDA**) by NVIDIA
    - High performance and easy to use but vendor lock-in
  - Open Computing Language (OpenCL)
    - Open standard, cross-platform but complex and varying performance
    - Microsoft DirectCL (a wrapper for OpenCL)
  - Programming languages
    - Python, C/C++, wrappers for many other languages
- **Deep learning frameworks**
  - PyTorch, TensorFlow
- **Hybrid parallel programming frameworks**
  - CUDA + MPI



GPU Computing

Beyond the CPU



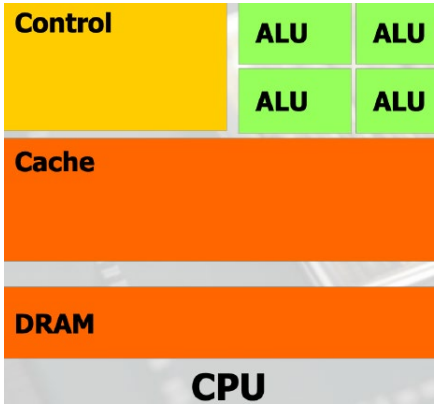
- **The motivation**
  - Computer graphics, games, deep learning, and many scientific simulations require parallel computation.
    - Vector and matrix operations for image processing
    - Weights learning in neural networks
    - These **operations** are inherently parallel.
  - The solution was to increase the computational speed of CPU,
  - but single-core CPU performance has essentially stagnated.
  - Better solution was to increase parallelism with multicore,
  - but multi-core CPUs are expensive.
  - Even better solution is to add massively parallel, cheap, and programmable accelerators => **GPUs**.

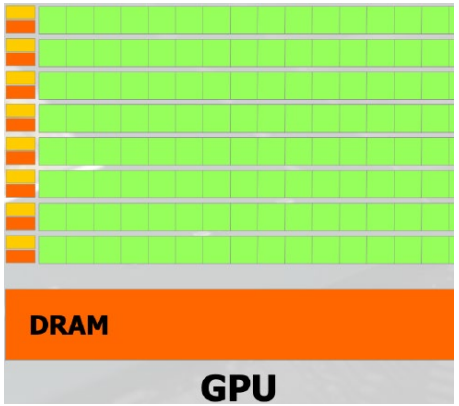
- **The rise of the Graphics Processing Unit (GPU)**
  - The term first popularized by NVIDIA in 1999 with the world's first GPU, **GeForce 256** video card.
    - Originally built for graphics processing, hardwired logic, little to no programmability
    - NVIDIA released "**CUDA**" software framework in **2006**, allowing fully programmable GPUs for non-graphics computations.

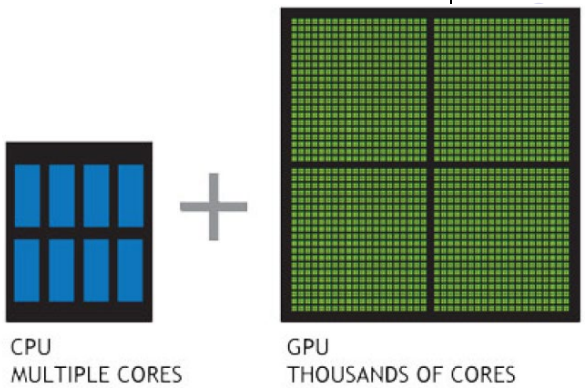
GPUs Today

- **Why GPU?**
  - Relatively cheap many (thousands) core chips for massively multithreaded processing
  - Cores for General Purpose Computing (CUDA cores), special cores (tensor, RT cores)
- **Two major vendors**
  - NVIDIA:
    - GeForce RTX series (gaming), Quadro/RTX series (workstation)
    - Hxx, Axx, Lxx series (servers, data centers)
  - AMD: Radeon series, FirePro series
- **GPUs today**
  - GPUs are so successful and widely deployed as accelerators.
    - NVIDIA GeForce RTX 5090: 80 teraflops with 18,432 cores, 32GB VRAM
    - NVIDIA H100: 14,592 CUDA cores, 3,959 teraflops by SXM with 528 FP8 tensor cores (for matrix multiplication and addition)
    - **NVIDIA L40s Ada** with 4 GPUs (**CS dept. in 2024**): 18,176 CUDA cores, 568 tensor cores, 142 RT cores, 48GB memory, 10TB storage, Intel C741 chip, 1,466 teraflops for FP8
  - Other CPU alternatives are dead.

How is GPU different from CPU?





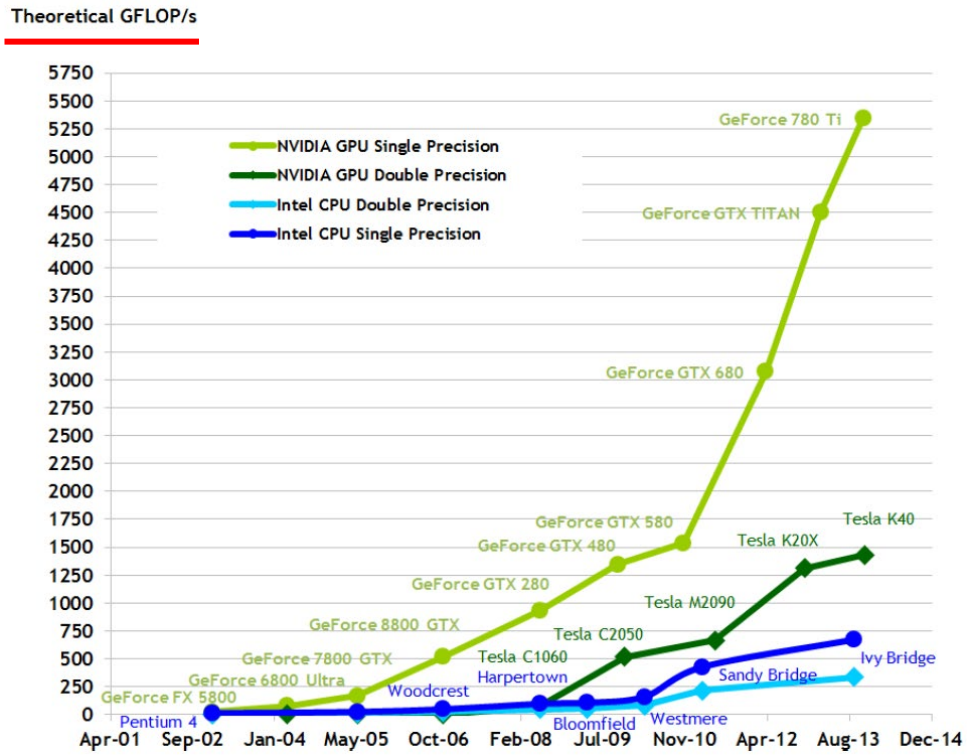


A computer with GPUs

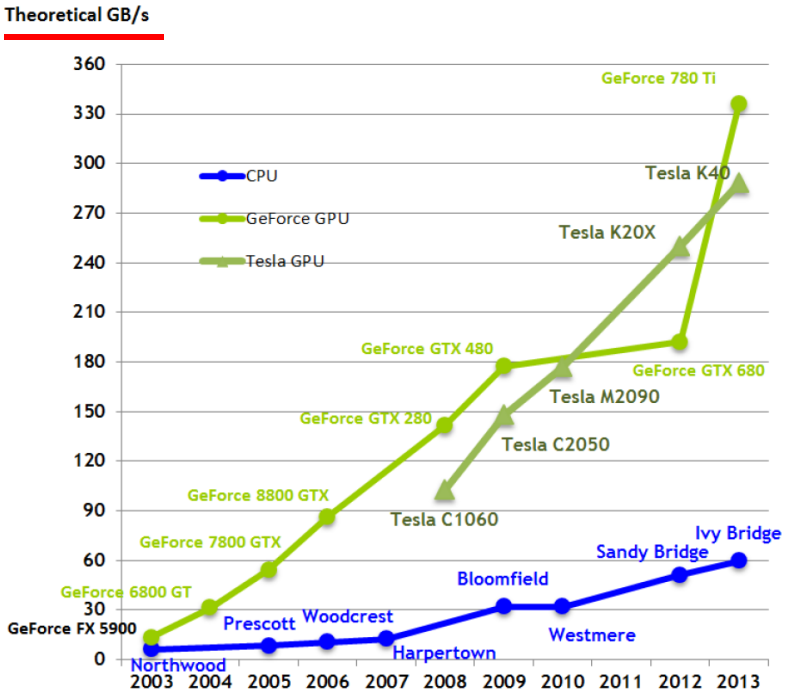
- **CPU** (**host**)
  - Few processing cores but highly flexible for general purpose
  - Low latency and moderate throughput
    - Latency: the time it takes for a task or data to complete one cycle
    - Throughput: the amount of work or data processed by a system in a given time
- **GPU** (**device**, accelerator card)
  - Its own memory and coprocessor connected to the CPU (host)
  - Many processing cores but limited flexibility for high latency and high throughput with high parallel computation using many threads
  - Shared instruction streams across many fragments

Computing and Memory Bandwidth

Computing Performance

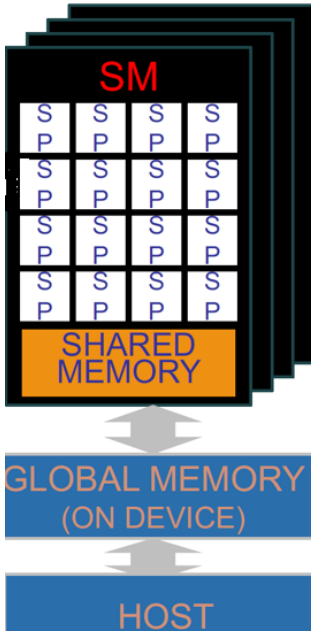


Memory Bandwidth

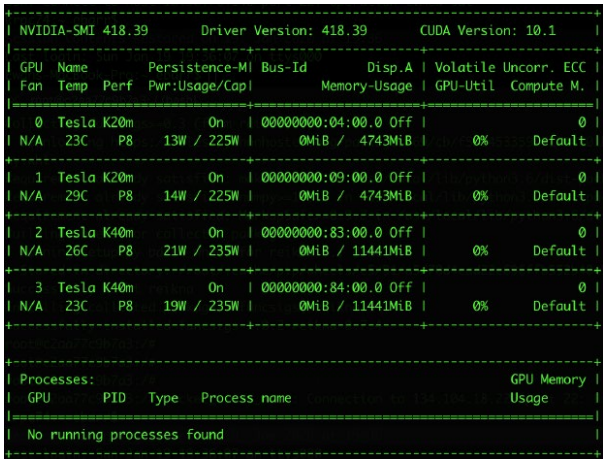


GPU Cores

- **Scalar processor and vector processor**
  - Scalar processors: processors that process only one data item at a time.
  - Vector processors: processors that implements instructions operating on an array of data (SIMD).
- **Streaming Multiprocessors** (SMs or SMX)
  - 32 (or 16, 48 or more)x Scalar Processor (SP)
    - "**CUDA core**" in NVIDIA or "Stream Processor" in AMD) with fast local "shared memory" between SPs
    - CUDA core is basic unit like SP to execute an instruction per a clock cycle but by SIMT for flexibility and efficient memory access.
  - For Single Instruction, Multiple Thread (SIMT) in parallel
    - Executes one instruction in multiple threads independently.
- **Tensor cores**
  - One instruction process matrices/tensors, e.g., (multiplication + addition) in parallel, like vector processor
- **Shader cores, RT cores** for graphics processing

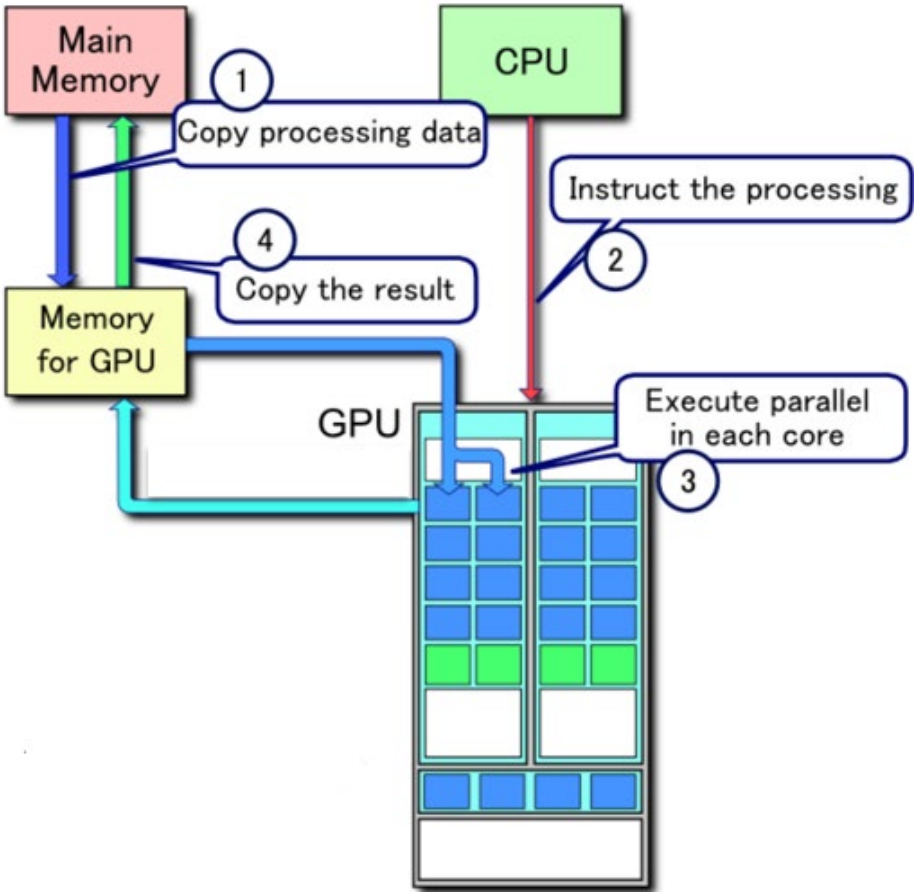


CUDA (Compute Unified Device Architecture)

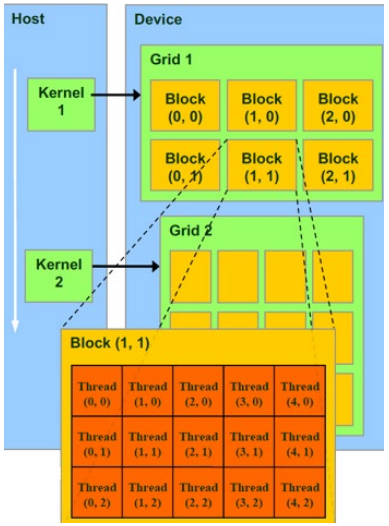


- **What is CUDA?**
  - First GPU programming framework that allow general-purpose programming with NVIDIA GPUs
    - not supported by other accelerators
- **CUDA language, device API, and runtime API**
  - C++ dialect (special language extensions for GPU code) that allows the host (CPU) code and GPU code in the same file
  - CUDA device API is low-level API that CUDA runtime API is built upon.
  - CUDA runtime API manages runtime GPU environment, allocates memory, data transfers, synchronization with GPU, etc. usually invoked by host code.
- **CUDA installation**
  - Download **CUDA framework** from "developer.nvidia.com".
    - Driver, toolkit (compiler nvcc), SDK, programmers guide
    - Other tools: emulator (executes on CPU, slow), cuda-ddb (Linux)
  - Verify GPU: type "**nvidia-smi**" at command prompt.

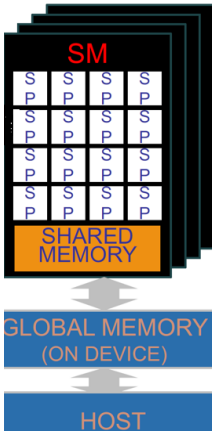
Processing Flow on CUDA



CUDA Core Execution

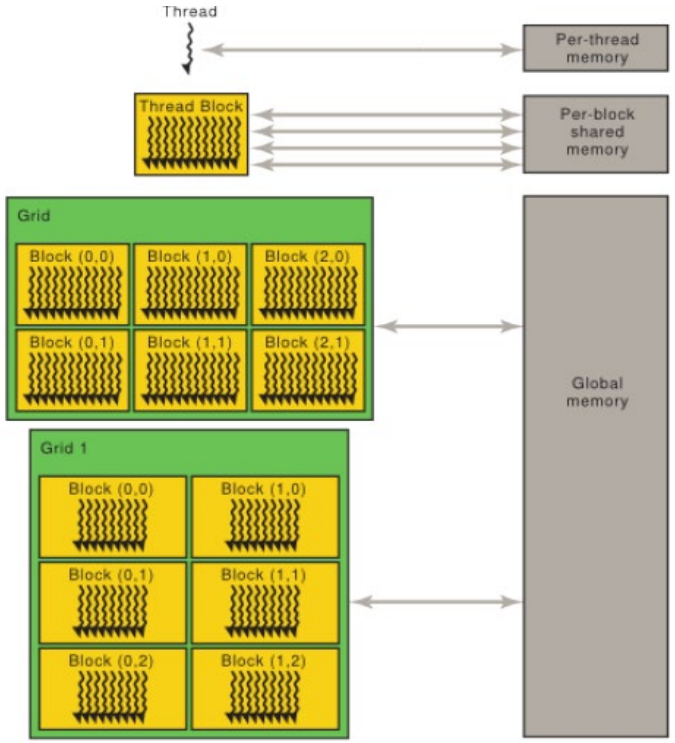


- **Single-thread execution**
  - A CUDA core can execute the instructions for a single *thread*.
    - GPU thread is extremely lightweight compared to the CPU thread.
- **Warp-level execution**
  - A warp is a group of 32 *threads* that are executed simultaneously.
    - If a SM has 64 CUDA cores, it can execute two warps concurrently.
    - All threads in a warp execute the same instruction (SIMD model).
- **Block-level execution**
  - A block is a group of warps (up to 32 warps).
    - Threads are grouped in blocks, e.g., 128, 192, or 256 threads in a block.
    - Thread ID is unique within block.
  - A **block** is executed on one **SM** (streaming multiprocessor).
    - Every block has its own shared memory and registers in the SM.
    - One thread block executes on one SM in parallel independently.
    - All threads sharing the low latency “shared memory” to synchronize their execution
    - Two threads from two different blocks cannot cooperate.

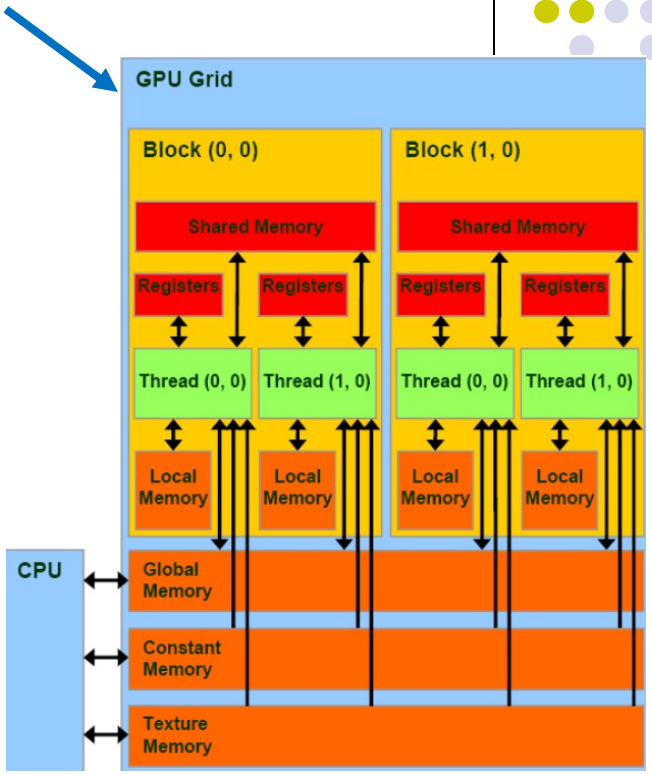


- **Grid-level execution**
  - A grid is a group of blocks. A kernel (program) is executed as **a grid** of blocks.

GPU Memory Hierarchy



Registers & local memory



- **Types of memory**
  - Global memory for I/O (large and slower)
  - Shared memory for thread collaboration (small and fast)
  - Registers for thread space (small and fastest)
  - Constant memory: 64 KB, global, for broadcast read-only data during execution
  - Texture memory: global, read-only access to 2D or 3D data (images)

GPU Programming Models and Tools

Table 1: GPU Computation APIs

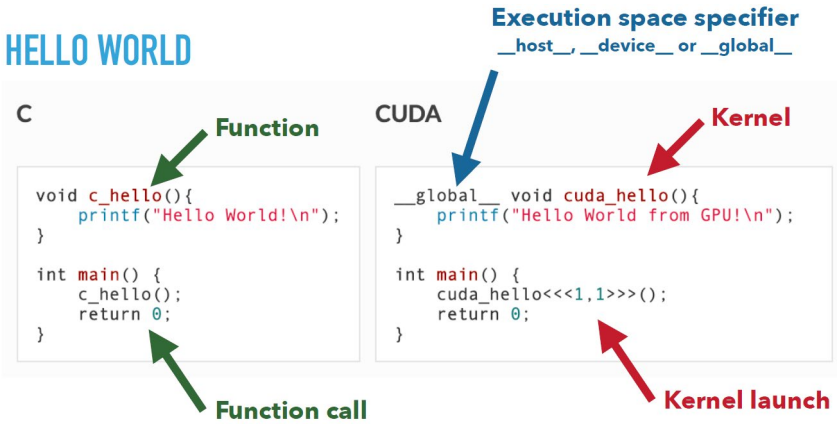
Name	Ease of Programming	Cross-Platform?	Performance (Guide)
CUDA	High	No (Hardware)	High
OpenCL	Medium	Yes	Medium-High
DirectCompute	Medium	No (Software)	Low
C++ AMP	Highest	No (Software)*	Lowest
GLSL Compute Shaders	Lowest	Yes	Highest

Increasing order of difficulty and flexibility

Model	GPU	CPU Equivalent
Vectorizing compiler	PGI CUDA FORTRAN	gcc, icc
“drop-in” libraries	cuBLAS	ATLAS
Directives	OpenACC, OpenMP-to-CUDA	OpenMP
High-level language	CuPy, NumBa, cuML, cuDNN	Python
Mid-level language	PyCUDA, OpenCL, CUDA API	Pthreads + C/C++
Low-level languages	PTX (NVIDIA intermediate)	Shader
Hardware-level language	SASS (NVIDIA assembly)	

CuPy: array-based computing (drop-in replacement for NumPy)
cuML, **cuDNN**: part of RAPIDS AI framework by NVIDIA
Numba: just-in-time (**JIT**) compiler at runtime for both CPU and GPU, also CUDA kernels
PTX:CUDA compiler translates to human readable intermediate code for portability
SASS (streaming assembly): hardware-level

CUDA Programming: Hello World



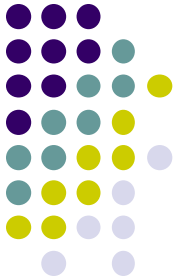
nvcc –o hello hello.cu

```python
# numba translates Python byte-code to machine code before execution
# to optimize CPU and CUDA GPU functions using decorators
from numba import cuda

# Define a CUDA kernel
@cuda.jit
def hello_world():
    # Get thread and block IDs
    thread_id = cuda.threadIdx.x
    block_id = cuda.blockIdx.x

    # Each thread prints "Hello, World!"
    print("Hello, World! From thread (", thread_id, ", ", block_id, ")")

# Configure the number of threads and blocks: 2x4=8 threads
threads_per_block = 4
blocks_per_grid = 2

# The GPU allocates resources to execute the kernel and launch the kernel
hello_world[blocks_per_grid, threads_per_block]()
```

python hello.py
Each of total 8 threads will print “**Hello, World!**”

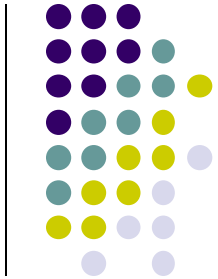
- **Host and Device code are asynchronous**.
  - Good because GPU can do work at the same time as CPU does work.
  - It is user’s responsibility to synchronize and to check errors.
  - CUDA kernel launches return immediately.

How to Utilize HPC



1. **Analyze the problem**
   - Determine parallelizability
   - Identify dependencies
   - Select granularity of tasks (many small tasks or few large tasks)
2. **Choose the programming model and language**
   - Message passing model, shared memory model, hybrid
   - OpenCL, CUDA, OpenMP, C++, Python, etc.
3. **Set up the environment**
   - Hardware requirements, programming tools, compiler, and libraries
4. **Design the program architecture**
   - Divide the workload, map tasks to processors
5. **Implement the code and debug**
   - Set up parallel constructs, distribute workload, implement synchronization
6. **Test and benchmark**
   - Verify correctness with small datasets
   - Test with larger datasets to check performance
7. **Optimize through iteration and refinement**
   - Analyze bottlenecks and optimize

CUDA Programming with PyTorch

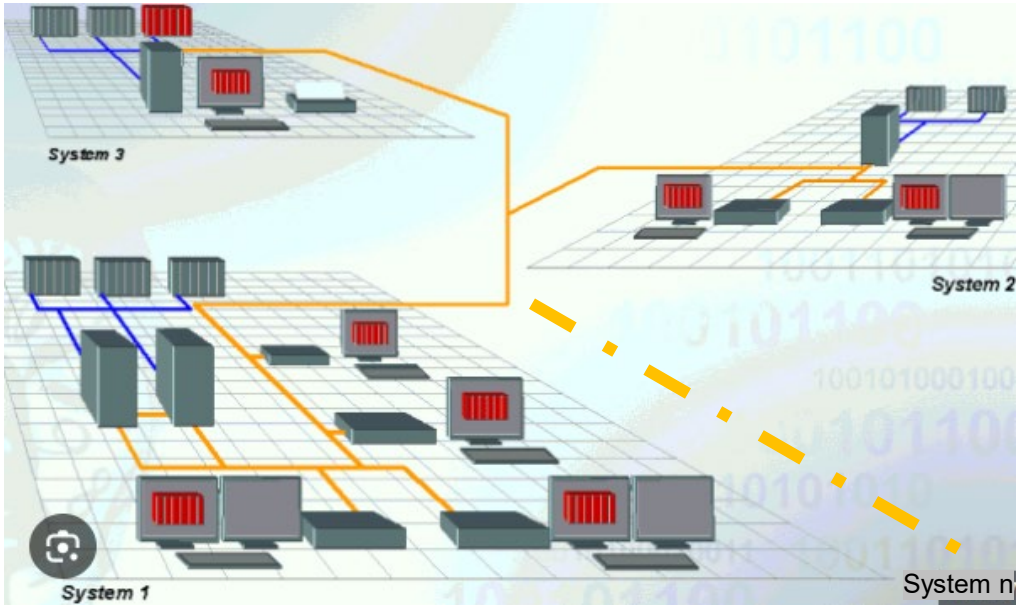


```python
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from torch.utils.data import DataLoader, TensorDataset

# Check if GPU is available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

# Load Iris dataset
iris = load_iris()
X = iris.data  # Features
```

```python
# Initialize the model, loss function, and optimizer
model = IrisNN(input_size, hidden_size, output_size).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

# Training loop
print("Training...")
for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
    for batch_X, batch_y in train_loader:
        # Move data to GPU
        batch_X, batch_y = batch_X.to(device), batch_y.to(device)
```

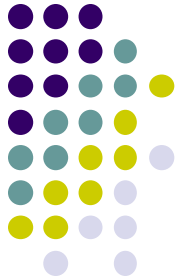
- **In PyTorch**
  - To automatically uses GPUs if available or fall back to the CPU
  - To use GPUs, check the availability of GPU device and send computation to the device
- **Data transfer between devices**
  - .to(device) to move tensors between devices
  - .cpu() to bring results back to the CPU for further processing

A geographically distributed clustered computing system



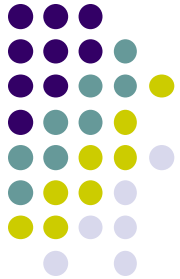
Distributed Clustered Computing Platform

The NRP and Nautilus Cluster



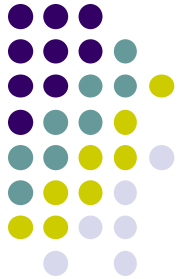
- **The National Research Platform (NRP)**
  - A partnership of more than 50 institutions
    - UC San Diego, Univ. of Nebraska-Lincoln, Massachusetts Green High Performance Computing Center, the National Science Foundation, the Department of Energy, the Department of Defense, Cal State Fullerton, etc.
- **What is the Nautilus?**
  - A distributed computing platform that connects various computing systems that offer computing resources such as **GPUs**, CPUs, FPGAs for research and education, across multiple locations
- **Why Nautilus?**
  - A hyper-cluster facilitating scalable computing for simulations and AI-related tasks
  - Plenty of shared computing resources contributed by any universities

The Nautilus Architecture

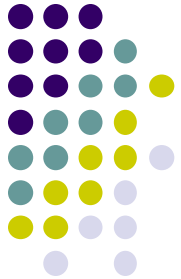


- **Distributed architecture**
  - Multiple data center and institutions, forming a geographically distributed system
  - Nodes are connected via high-speed networks
  - Popular frameworks
    - Hyper-converged Infrastructure by Nutanix, 2009 and private custom frameworks such AWS, Google, Azure
- **Hyper-Converged Infrastructure (HCI)**
  - combines computing nodes, distributed storage system, and networking
    - into a unified, and scalable system, allowing dynamic allocation of resources to meet the workload demands
  - provides the foundation for a distributed system by pooling and managing resources across multiple nodes, allowing private or hybrid cloud
- **Kubernetes (K8s) for container orchestration platform**
  - To manage containerized applications, enabling portability and efficient resource utilization
  - To deploy, scale, and manage containerized application across the cluster

Ways to Access the Nautilus



- **JupyterHub**
  - Open-source multi-user platform that provides a centralized hub for managing and hosting JupyterLab or Jupyter Notebook
    - Multi-user access, authentication, resource allocation, deployment (single servers, clusters (e.g., Kubernetes via Zero to JupyterHub), cloud platform (AWS, Azure, GCP)
    - JupyterLab and Jupyter Notebook is an open-source application that allows users to create, run and share programs using a browser interactively.
- **Kubernetes**
  - Open-source platform to automate **containerized** application deployment, scaling, management, and orchestration
    - Platform-agnostic that runs on any platforms, clouds
  - What is container and containerization?
    - A lightweight, standalone executable software package that includes everything needed to run an application in an isolated, consistent, and resource-efficient manner
    - Containerization is a way to package all the necessary files for application to deploy and run them.
  - Use cases for containerized application deployment
    - Microservices architecture, DevOps and CI/CD pipelines, cloud development

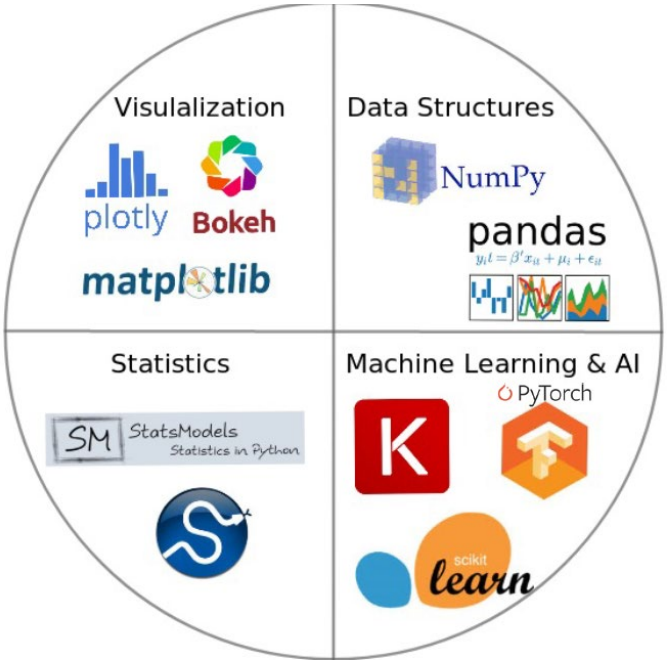


Machine Learning Packages and **Deep Learning Frameworks**

Machine Learning (ML) Packages and Tools

- **Major programming languages and AI or ML packages**
  - **Python**
    - Numpy, pandas, scikit-learn, PyTorch, TensorFlow, etc.
    - Agentic AI frameworks
  - **JavaScript (JS)/TypeScript (TS)**
    - ml.js (like Scikit-Learn in JS/TS), scikit-js, scikit-learn-ts, TensorFlow.js, js-pytorch
    - Agentic AI frameworks – what's the strength of JavaScript in Agentic AIfor AI agents?
  - Other languages
    - Microsoft C# and .NET: ML.NET, Excel
    - Java: Weka, Mahout, MOA
    - Non-CS users: MATLAB, R and statistics tools such as SPSS, SAS
- Platforms providing packages and services
  - Cloud computing service providers: AWS, Azure, Google
  - Hugging face
  - LangChain

Python Ecosystem

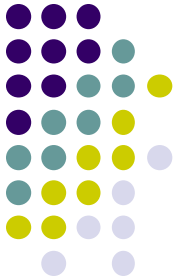


- A collection of tools, libraries, frameworks, and platforms
- Popularly used for (Web) applications, data science, statistics, machine learning, artificial intelligence, etc.

Python Packages for Machine Learning

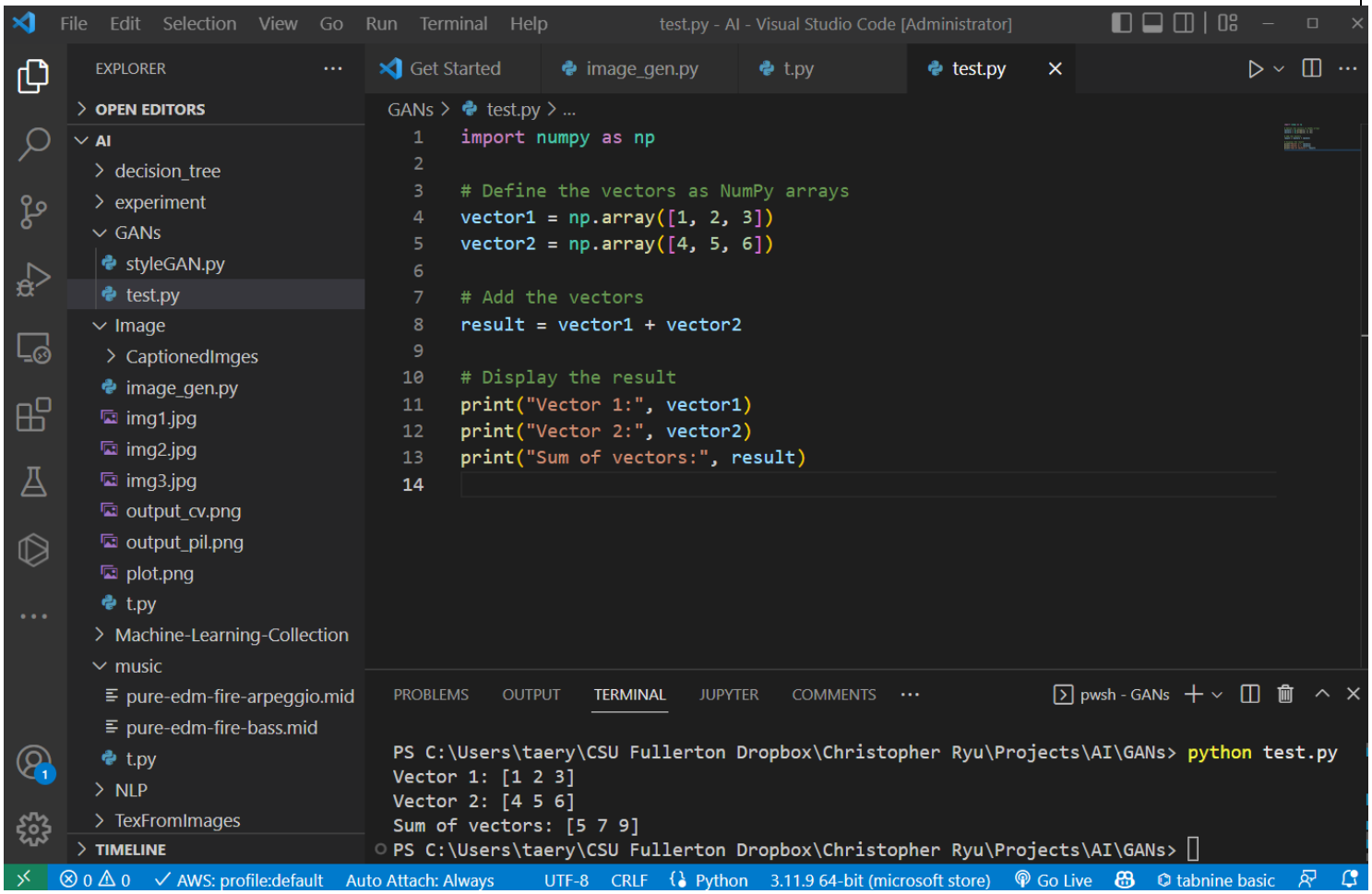
- **Numpy**: Fast numerical data processing and computation (>10x faster than Python)
  - https://numpy.org
- **Pandas**: Data loading, structuring, cleaning, searching, processing
  - https://pandas.pydata.org, https://www.w3schools.com/python/pandas/default.asp
- **Statistics**: Statistics libraries
  - https://docs.python.org/3/library/statistics.html
  - https://scipy-lectures.org/packages/statistics/index.html
- **Scipy**: Scientific computation
  - https://docs.scipy.org/doc/scipy/getting_started.html
- **Matplotlib**: Plotting libraries
  - https://matplotlib.org/stable/tutorials/introductory/pyplot.html
- **Scikit-learn**: ML packages using numpy, pandas, scipy, matplotlib
  - https://scikit-learn.org
- **PyTorch**: ANN libraries including deep learning and natural language processing
  - https://pytorch.org/tutorials/
- **TensorFlow** and **Keras**: Deep learning framework with APIs built on top of TensorFlow
  - https://keras.io/
- **Anaconda**: Python distribution including most packages
  - https://www.anaconda.com/

Setting Up Development Environment



- **Install an IDE tool**
  - such as **VS Code** (https://code.visualstudio.com/download)
- **Install Python and package manager**
  - Python from https://www.python.org/
    - Python package manager "**pip**" is bundled with Python interpreter.
  - Open a terminal or command prompt and type the following commands to verify
    - "python --version"
    - "pip --version"
- **Install the necessary Python packages as needed**
  - "pip install numpy"
  - "pip install pandas matplotlib"
  - or "pip install -r **requirements.txt**" to install multiple packages
- (*Optionally*) **setup virtual environment** (for easier containerization and deployment)
  - To create a virtual environment, type "python –m venv **myenv**" ("myenv" is the dir. name)
  - To activate it on Windows, type "**myenv**\Scripts\activate"
  - To activate it or macOS or Linux, type "source **myenv**/bin/activate"

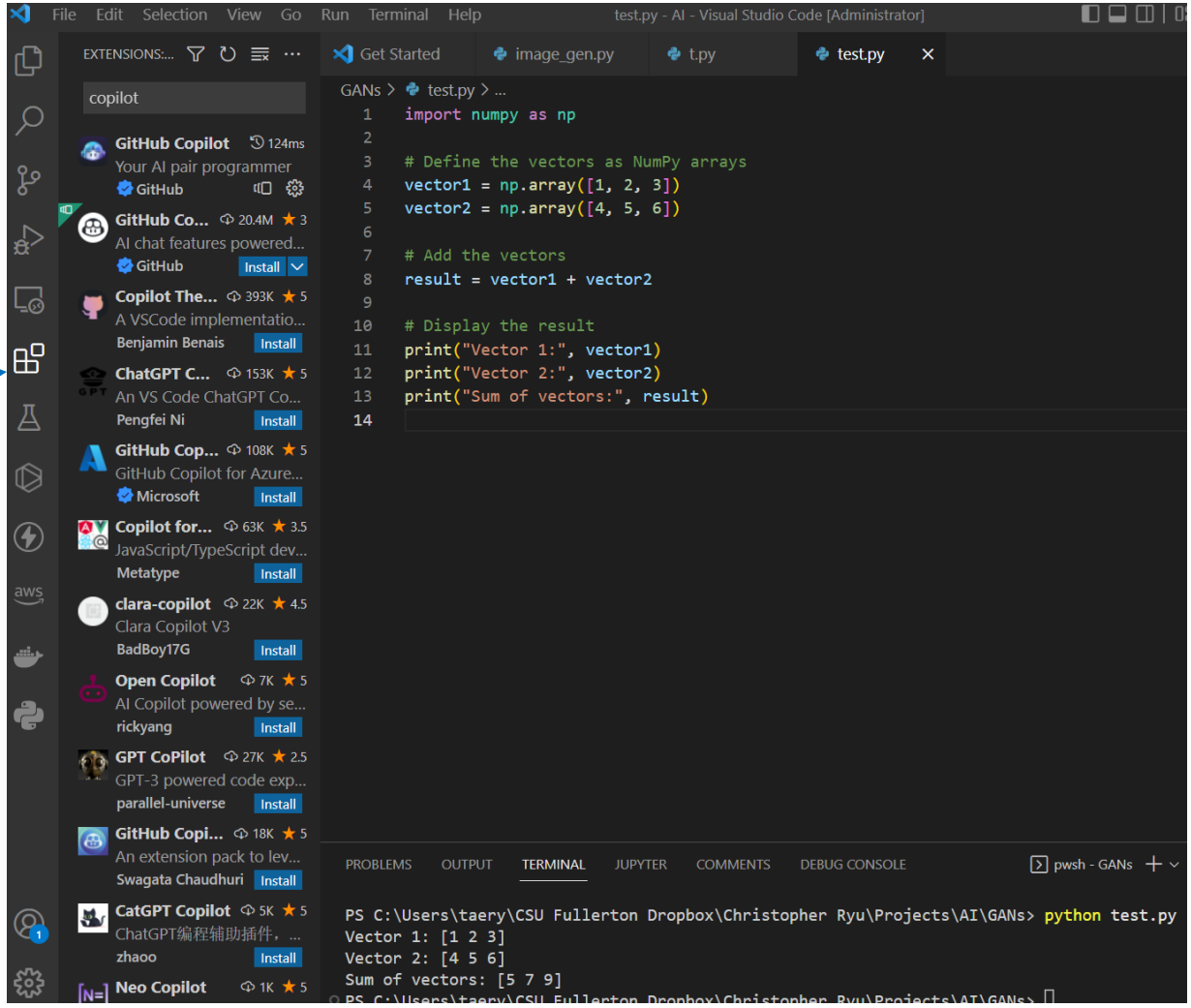
Python Programming with VS Code



Python Programming with VS Code

Source control

Adding extensions such as CoPilot or ChatGPT

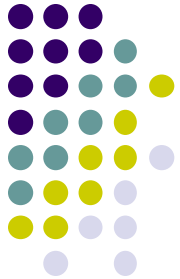


Example: Classification using scikit-learn

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load dataset
data = load_iris()
X, y = data.data, data.target

# Split into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train a classifier
clf = RandomForestClassifier(random_state=42)
clf.fit(X_train, y_train)

# Make predictions
y_pred = clf.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy:.2f}")
```

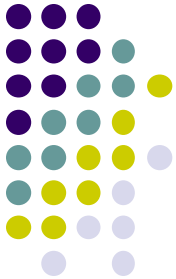
- Classifying the “iris” dataset using Random Forest algorithm

Python Documentation and Tutorial



- Official Python documentation
  - https://docs.python.org
- Official NumPy documentation
  - https://numpy.org/doc/stable/
- Official Scikit-learn documentation
  - https://scikit-learn.org/stable/
- Some **tutorial websites**
  - https://docs.python.org/3/tutorial/index.html
  - https://www.w3schools.com/python/
  - Plenty of YouTube tutorials available

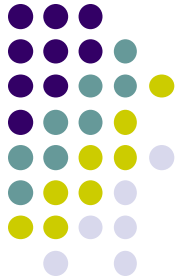
How to Declare Vectors and Matrices



- **NumPy** (for the Python package for scientific computing)
  - Multi-dimensional array object to store homogeneous or heterogenous data and optimized functions to operate on this array object
  - **Broadcasting in NumPy**
    - A mechanism allowing arithmetic operations on arrays with different shapes.
    - Under certain conditions, the smaller array is "stretched" or "replicated" to match the shape of the larger array, allowing element-wise operations without explicit looping is useful.
    - Example, x = np.random.random((3,4)) and y = np.random.random((3,1))

      x + y will perform element-wise addition, adding y to each column of x.
  - **For efficient NumPy code**
    - Avoid explicit for-loops over indices at all costs!
    - for-loops will dramatically slow down the code (~10 – 100x).
- Other Python packages
  - Scikit-learn
  - PyTorch, TensorFlow

ndarray in NumPy



One-dimensional Array (1-D Array)

0	1			n-1
1	2	3	4	5

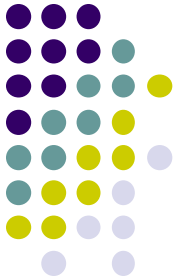
Two-dimensional Array (2-D Array)

	0	1			n-1
0	1	2	3	4	5
1	6	7	8	9	10
	11	12	13	14	15
m-1	16	17	18	19	20

```
import numpy as np

a = np.array([1,2,3,4,5])
```

```
a = np.array([[1,2,3,4,5],[6,7,8,9,10],[11,12,13,14,15],[16,17,18,19,20]])
```

For-loop and NumPy Code



With for-loop

```
for i in range(x.shape[0]):
    for j in range(x.shape[1]):
        x[i,j] **= 2
for i in range(100, 1000):
    for j in range(x.shape[1]):
        x[i, j] += 5
```

Without for-loop

This code is much faster!

```
x **= 2
```

```
x[np.arange(100,1000), :] += 5
```

- **Dot product**
  - result = np.dot(a, b)
- **Matrix-matrix product**
  - A = np.random.rand(1000,1000)
  - B = np.random.rand(1000,1000)
  - C = A @ B

For-loop and NumPy Code

Code using for-loop

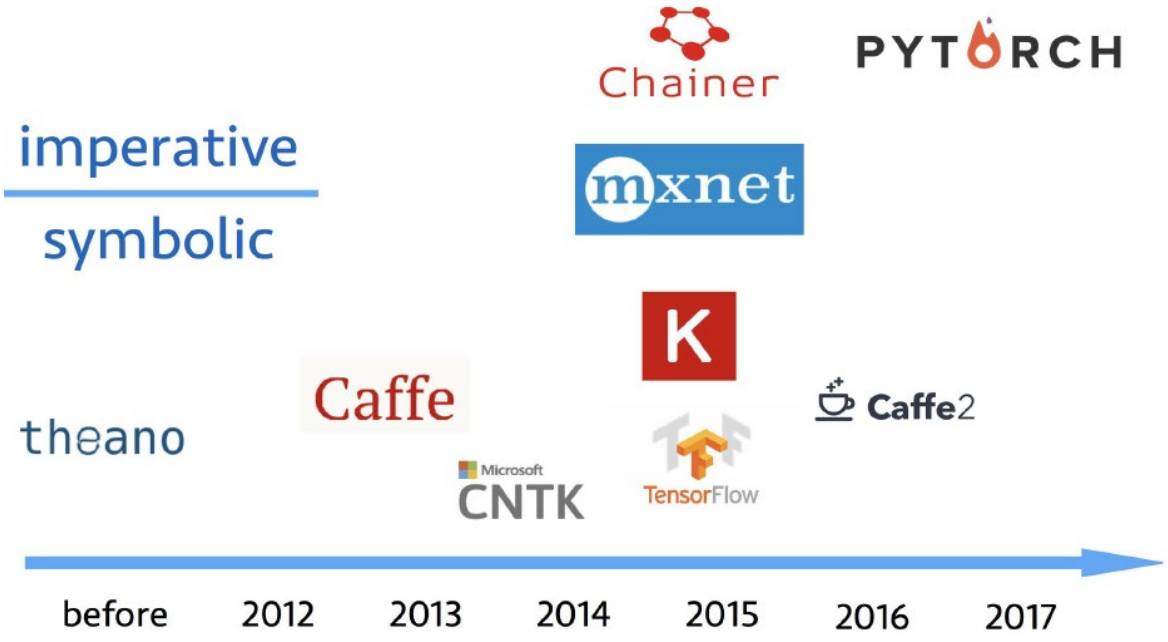
```
for i in range(x.shape[0]):
    for j in range(x.shape[1]):
        x[i,j] **= 2
for i in range(100, 1000):
    for j in range(x.shape[1]):
        x[i, j] += 5
```

Code without using for-loop

This code is much faster!

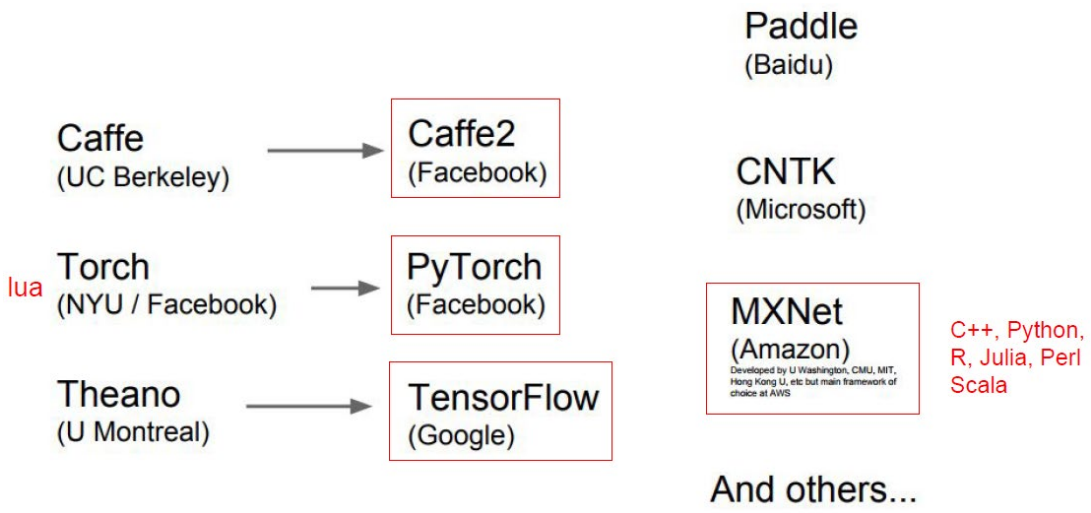
```
x **= 2
x[np.arange(100,1000), :] += 5
```

Deep Learning Frameworks



- **Deep learning programming frameworks**
  - Open-source based on artificial neural networks
  - Leverage the power of GPUs
  - Automatic computation of gradients that makes it easier to test and develop new ideas

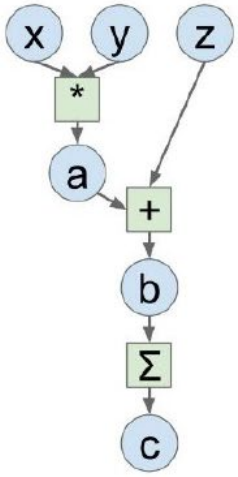
Popular Deep Learning Frameworks



- **Python-native** (optimized for mathematical computations in C++)
  - Theano, PyTorch, Keras
  - cuDNN: a lower-level C++ library (by NVIDIA, not a framework) to accelerate deep learning operations (convolution, pooling, batch normalization, RNNs, activation functions) on NVIDIA GPUs.
    - PyTorch uses cuDNN for operations
- **C++-based with API** (for Python and other languages)
  - Caffe, Tensorflow, MXNet, CNTK

Why PyTorch?

Computation Graph



Numpy

```
import numpy as np
np.random.seed(0)

N, D = 3, 4

x = np.random.randn(N, D)
y = np.random.randn(N, D)
z = np.random.randn(N, D)

a = x * y
b = a + z
c = np.sum(b)

grad_c = 1.0
grad_b = grad_c * np.ones((N, D))
grad_a = grad_b.copy()
grad_z = grad_b.copy()
grad_x = grad_a * y
grad_y = grad_a * x
```

Tensorflow

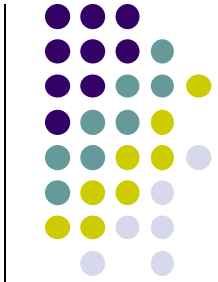
```
import numpy as np
np.random.seed(0)
import tensorflow as tf

N, D = 3, 4

with tf.device('/gpu:0'):
    x = tf.placeholder(tf.float32)
    y = tf.placeholder(tf.float32)
    z = tf.placeholder(tf.float32)

    a = x * y
    b = a + z
    c = tf.reduce_sum(b)

grad_x, grad_y, grad_z = tf.gradients(c, [x, y, z])

with tf.Session() as sess:
    values = {
        x: np.random.randn(N, D),
        y: np.random.randn(N, D),
        z: np.random.randn(N, D),
    }
    out = sess.run([c, grad_x, grad_y, grad_z],
                   feed_dict=values)
    c_val, grad_x_val, grad_y_val, grad_z_val = out
```

PyTorch

```
import torch

N, D = 3, 4

x = torch.rand((N, D),requires_grad=True)
y = torch.rand((N, D),requires_grad=True)
z = torch.rand((N, D),requires_grad=True)

a =x * y
b =a + z
c=torch.sum(b)

c.backward()
```

- **Computation graph**
  - The flow of computations and dependencies between operations
  - **Node** represents functions, variables, and operations to facilitate "forward" computation (outputs) and "backward" computation (backpropagation)
- **Installation**
  - **Anaconda/miniconda**: conda install pytorch torchvision torchaudio cudatoolkit=11.8 –c pytorch
  - **pip**: pip3 install torch torchvision torchaudio cudatoolkit=11.8

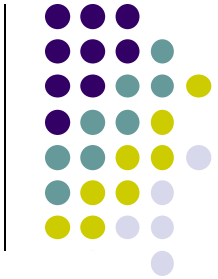
Key Elements for Deep Learning



- **Tensors** (torch.tensor)
  - Core data structure like Numpy arrays, but runs on GPU and autograd
    - Common operations are similar to those for ndarrays, e.g., rand, ones, zeros, indexing, slicing, reshape, transpose, element wise multiplication, matrix product
    - Tensors are created using torch.tensor().
- **Autograd** (torch.autograd)
  - Tensors for computations and automatic differentiation
    - .backward() triggers gradient computation
    - .grad stores computed gradients
- **Module** (torch.nn)
  - A neural network layer that may store state or learnable weights.
    - Predefined layers: nn.Linear, nn.Conv2d, nn.ReLU, etc.
    - Loss functions: nn.CrossEntropyLoss, nn.MSELoss
    - Optimizers: torch.optim

```
# Create tensors.
x = torch.tensor(1., requires_grad=True)
w = torch.tensor(2., requires_grad=True)
b = torch.tensor(3., requires_grad=True)

# Build a computational graph.
y = w * x + b    # y = 2 * x + 3

# Compute gradients.
y.backward()

# Print out the gradients.
print(x.grad)    # x.grad = 2
print(w.grad)    # w.grad = 1
print(b.grad)    # b.grad = 1
```

Tensors on GPU



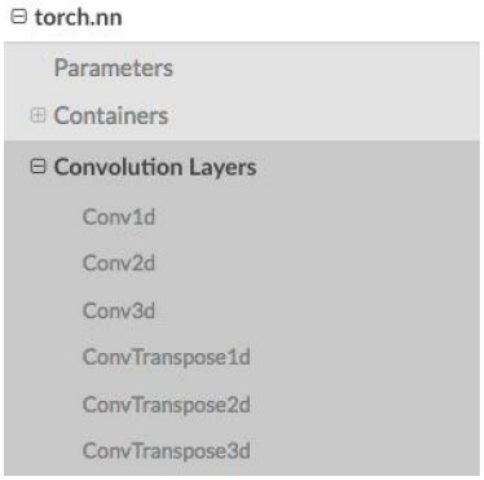
- PyTorch tensors can run on GPU.
- To run on GPU, just cast tensors to a CUDA datatype.
- To create random tensors for data and weights
- Forward pass: computing predictions and loss
- Backward pass: *manually* computing gradients
- Gradient descent step on weights

Two-layer network using tensors

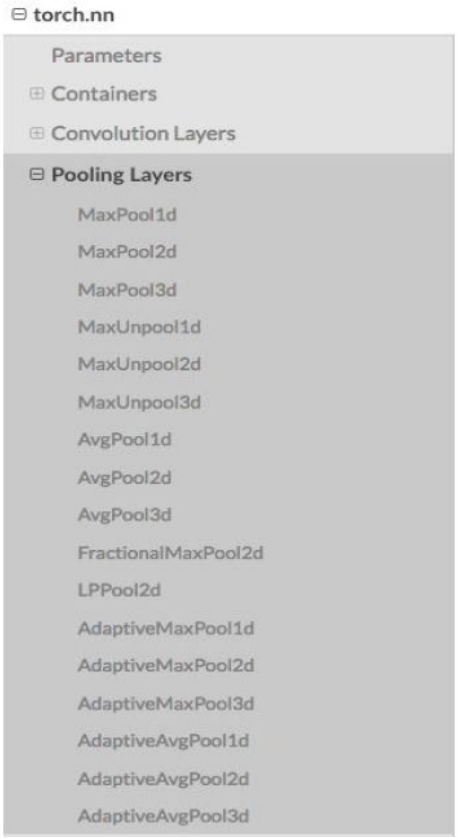
```
import torch

dtype = torch.cuda.FloatTensor

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in).type(dtype)
y = torch.randn(N, D_out).type(dtype)
w1 = torch.randn(D_in, H).type(dtype)
w2 = torch.randn(H, D_out).type(dtype)

learning_rate = 1e-6
for t in range(500):
    h = x.mm(w1)
    h_relu = h.clamp(min=0)
    y_pred = h_relu.mm(w2)
    loss = (y_pred - y).pow(2).sum()

    grad_y_pred = 2.0 * (y_pred - y)
    grad_w2 = h_relu.t().mm(grad_y_pred)
    grad_h_relu = grad_y_pred.mm(w2.t())
    grad_h = grad_h_relu.clone()
    grad_h[h < 0] = 0
    grad_w1 = x.t().mm(grad_h)

    w1 -= learning_rate * grad_w1
    w2 -= learning_rate * grad_w2
```

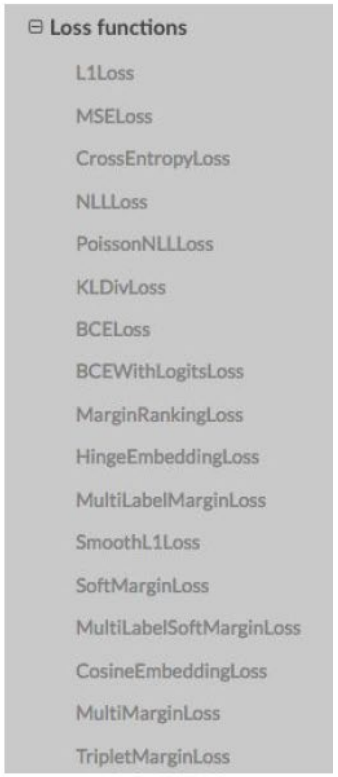
PyTorch: Module

- A neural network layer; may store state or learnable weights
  - torch.nn is the higher-level wrapper to work with neural networks, similar to Keras, but only one.
- Various layers: Dropout, Linear, Normalization, Convolution, Pooling

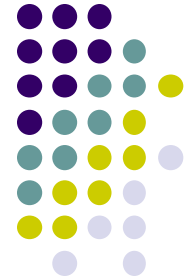


Module allows creating various neural networks.

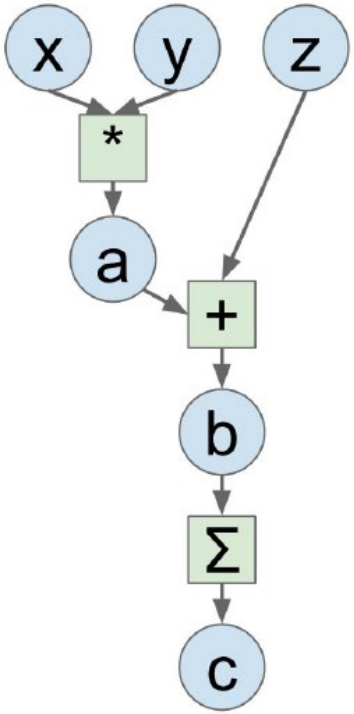




Computational Graphs with NumPy

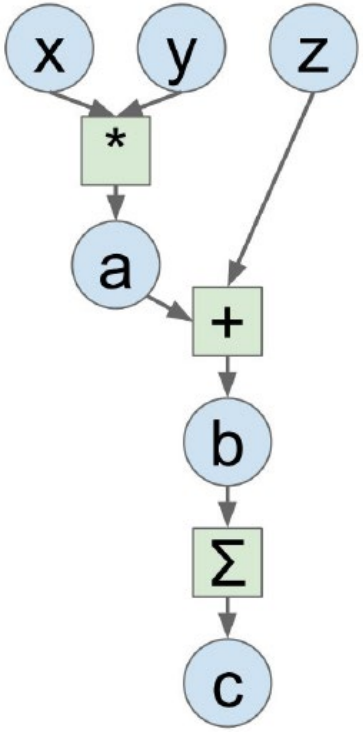


- **Why computational graph?**
  - The flow of computations and dependencies between operations
  - **Node** represents functions, variables, and operations to facilitate "forward" computation (outputs) and "backward" computation (backpropagation), essential in deep learning




```
import numpy as np
np.random.seed(0)

N, D = 3, 4

x = np.random.randn(N, D)
y = np.random.randn(N, D)
z = np.random.randn(N, D)

a = x * y
b = a + z
c = np.sum(b)

grad_c = 1.0
grad_b = grad_c * np.ones((N, D))
grad_a = grad_b.copy()
grad_z = grad_b.copy()
grad_x = grad_a * y
grad_y = grad_a * x
```

Computational Graphs with PyTorch



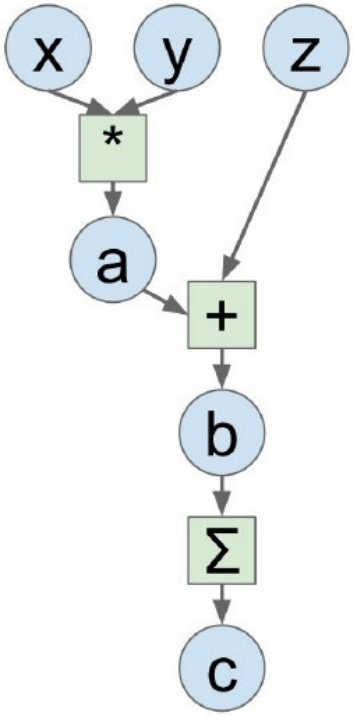
Defining **Variables** to start building a computational graph

Forward pass looks just like NumPy

Backward pass by calling c.backward() computes all gradients

```
import torch

N, D = 3, 4

x = torch.randn(N, D, requires_grad=True)
y = torch.randn(N, D, requires_grad=True)
z = torch.randn(N, D, requires_grad=True)

a = x * y
b = a + z
c = torch.sum(b)

c.backward()

print(x.grad.data)
print(y.grad.data)
print(z.grad.data)
```

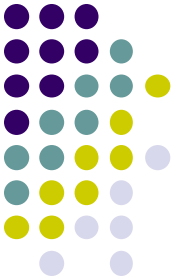
Computational Graphs with PyTorch



To run on GPU by sending tensor to the CUDA device

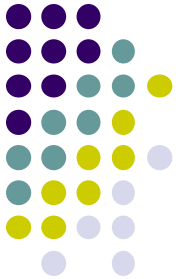
```
import torch

N, D = 3, 4

device = torch.device("cuda" if torch.cuda.is_available else "cpu")
x = torch.randn(N, D, requires_grad=True).to(device)
y = torch.randn(N, D, requires_grad=True).to(device)
z = torch.randn(N, D, requires_grad=True).to(device)

a = x * y
b = a + z
c = torch.sum(b)

c.backward()

print(x.grad.data)
print(y.grad.data)
print(z.grad.data)
```

Example Neural Network using PyTorch



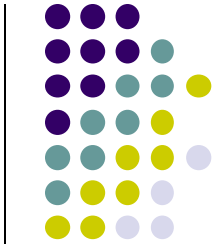
```python
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')  # Use GPU if available

# Define the neural network architecture
class SimpleNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(SimpleNN, self).__init__()
        # Define the layers
        self.fc1 = nn.Linear(input_size, hidden_size)  # First layer
        self.fc2 = nn.Linear(hidden_size, output_size)  # Second (output) layer
        self.sigmoid = nn.Sigmoid()  # Sigmoid activation for output

    def forward(self, x):
        x = torch.sigmoid(self.fc1(x))  # Apply sigmoid to the first layer
        x = self.sigmoid(self.fc2(x))   # Apply sigmoid to the second layer
        return x

# Generate random data (2 input features for binary classification)
np.random.seed(42)
X = np.random.randn(1000, 2).astype(np.float32)  # 1000 samples, 2 features
y = (np.random.randn(1000, 1) > 0).astype(np.float32)  # Binary labels (0 or 1)

# Convert data to PyTorch tensors and move to GPU
X_tensor = torch.tensor(X).to(device)
y_tensor = torch.tensor(y).to(device)

# Initialize the model
model = SimpleNN(2, 4, 1).to(device)
```

This example is an *incomplete implementation* of a neural network model.



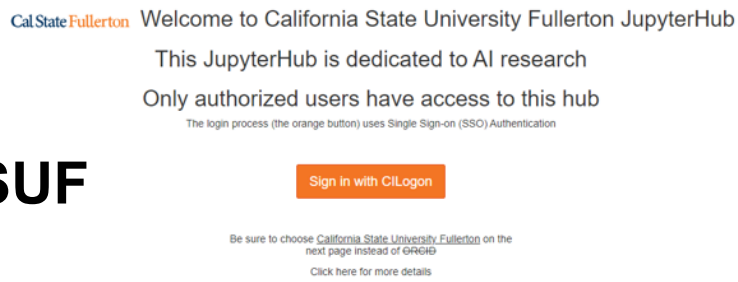
Deployment via JupyterHub

JupyterLab



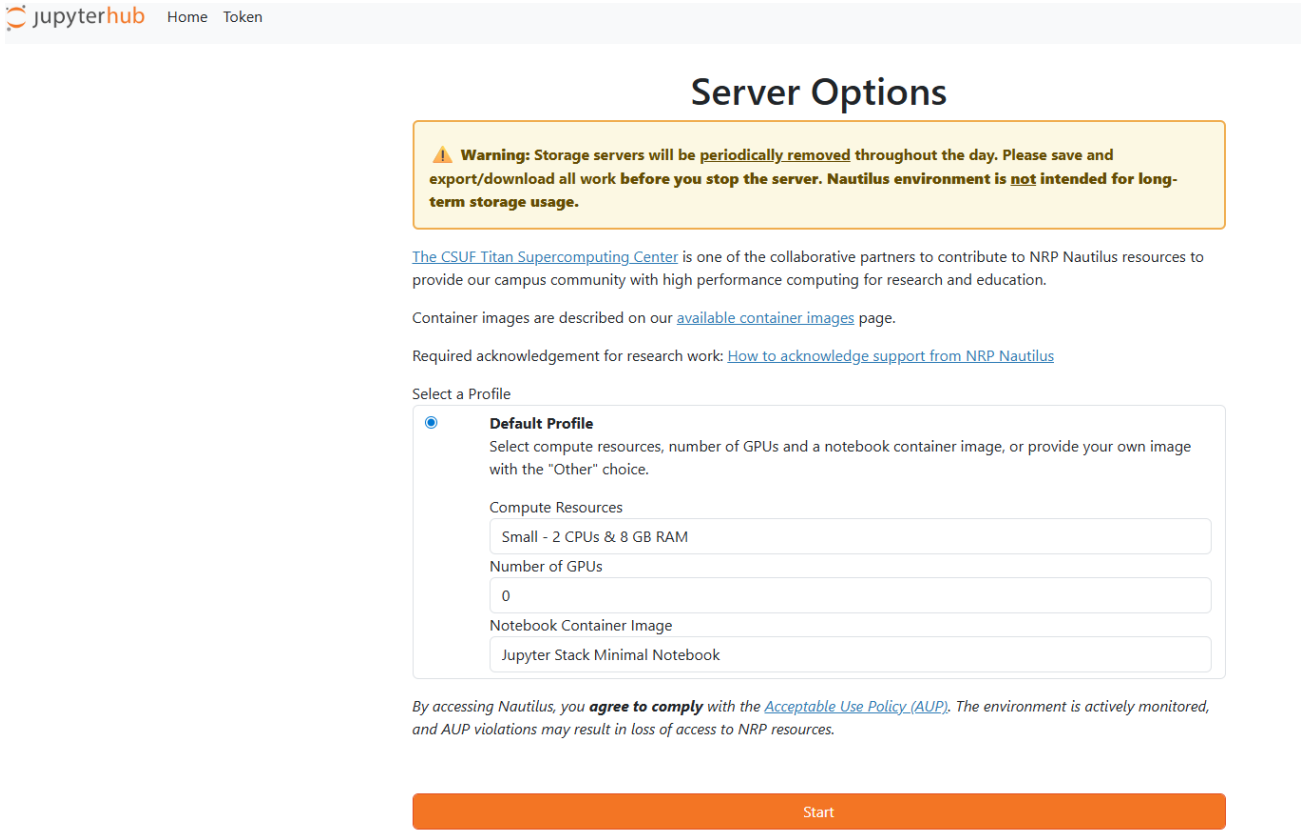
- **Jupyter** stands for Julia, Python, and R (now supports many other languages)
  - Jupyter Notebook vs JupyterLab
    - IPython (Python shell, 2008) => Jupyter Notebook (2014) => **JupyterLab** (2018)
  - Various JupyterLab elements
    - Kernel runs the user's code (Python is default).
    - Dashboard helps users manage Noteboks.
    - Terminal for command line
    - Console (e.g., Python)
    - File types: Notebook (.ipynb), Python File (.py), Text File (.txt), Markdown File (.md for documentation, READMEs)
- **Programming with JupyterLab**
  - Code cell
    - To write and execute code (using the kernel). You can enter multiple lines of code in one cell and execute them separately from other cells.
  - Markdown cell
    - To write formatted text and documentation using Markdown (like HTML markup)
  - Using JupyterLab locally (without connecting to the server)
    - "pip install jupyterlab"
    - "jupyter lab" to **start**

Accessing the Nautilus via JupyterHub



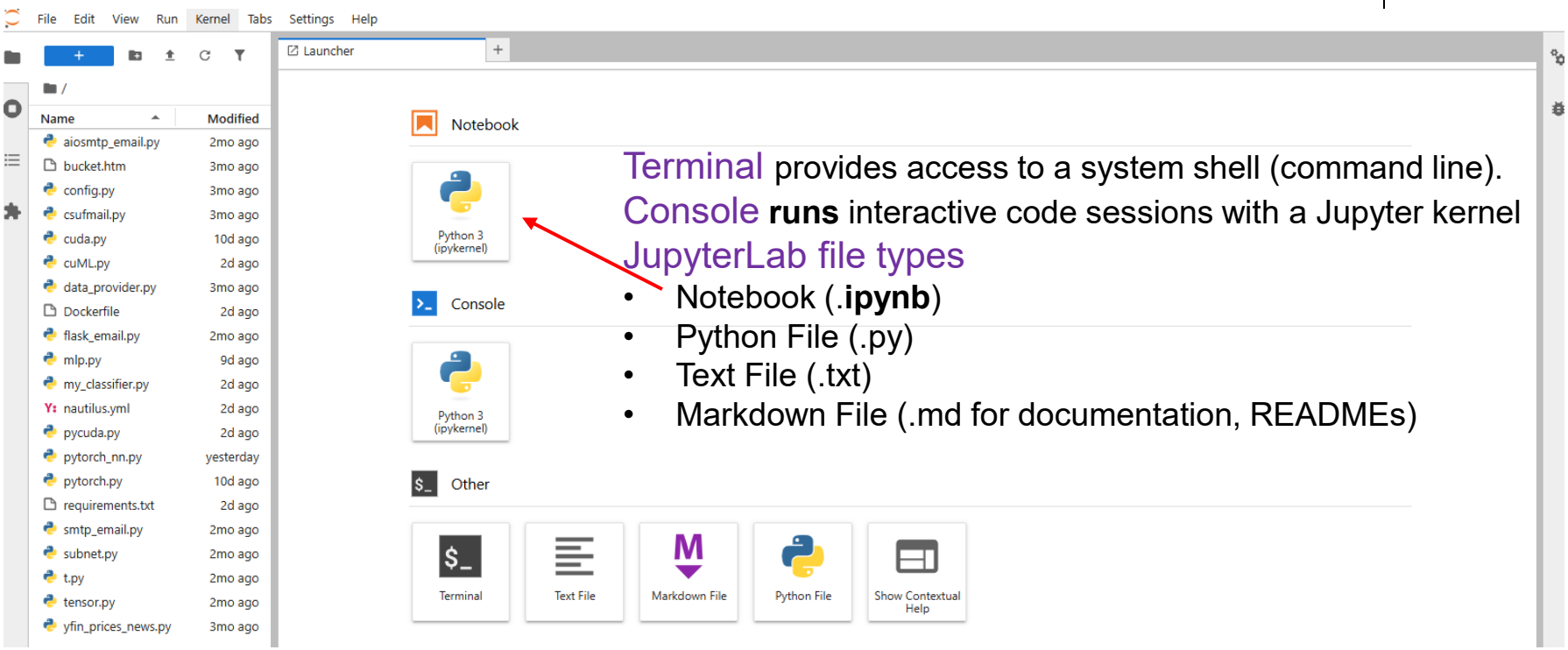
1. **Open the JupyterHub page for CSUF**
   - https://csuf-titans.nrp-nautilus.io
2. **Sign in with CILogon using your CSUF credential**
3. **Choose the computing resources and the package image**
   - Computing resources needed (e.g., GPU) and the image for the deployment package (e.g., PyTorch) to run your programs
   - Available resources can be found at the NRP site.
4. **Edit your program using the Jupyter Notebook or JupyterLab and execute it by clicking "run" button**
   - You can save the current workspace to a file
5. **Check the execution results**

Accessing the Nautilus via JupyterHub



- Choose CPU and GPU resources
- Choose a container image

JupyterLab IDE

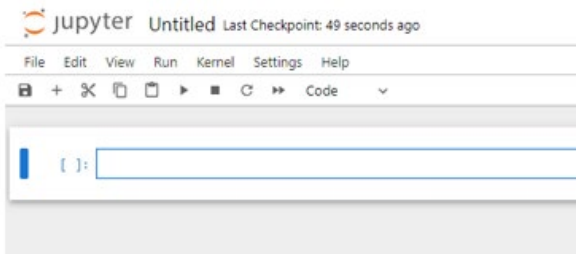


Terminal provides access to a system shell (command line).

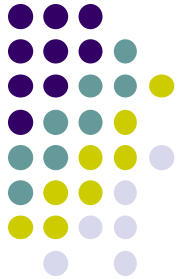
Console **runs** interactive code sessions with a Jupyter kernel

JupyterLab file types

- Notebook (.**ipynb**)
- Python File (.py)
- Text File (.txt)
- Markdown File (.md for documentation, READMEs)

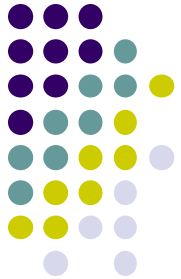


JupyterLab: Pros and Cons



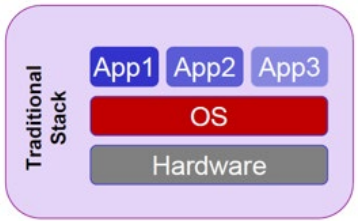
- **Benefits**
  - Easier interactive coding and testing
  - Sharing a Jupyter Notebooks and data
    - Cloud storage such as GitHub, Google drive, Dropbox, or OneDrive
    - JupyterHub for collaborative environment
    - Sharing via JupyterLab extensions (by enabling the feature and inviting collaborators)
  - Great for education and research collaboration

- **Limitations**
  - Complex software development
    - Collaboration using version control is common industry practice.
  - Efficiency of programs
  - Deploying complex applications



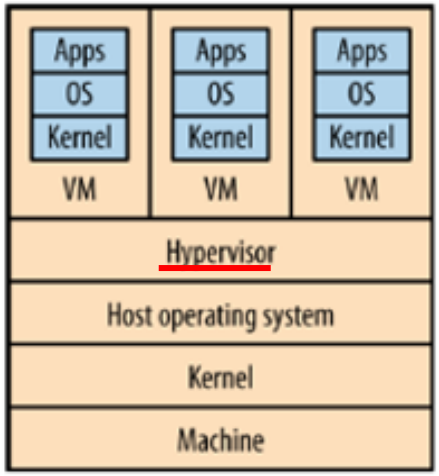
Deploying *Containerized* Applications via *Kubernetes*

Virtualization of Computing Resources



- **Sharing computing resources**
  - CPU (thread-scheduling mechanism)
  - Memory (RAM)
  - Disk storage (disk controller)
  - Network connections (identification of messages)
- **Hypervisor** (or similar intermediary)
  - runs on the host OS and manages the VMs like an operating system for VMs

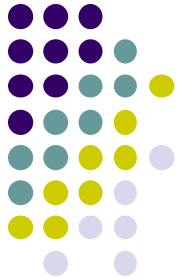
Host computer



Standard virtualizations

- **Virtual Machine (VM)**
  - A guest computer with its own operating system (OS).
    - A VM image is a file including the instructions to boot a VM as it bundles with all of its dependencies such as a boot loader, operating system, a root file system, and other services.
- **Implications of VMs for architects**
  - **Benefit**: Separation of concerns as VMs allows architects to treat runtime resources as commodities
  - **Tradeoff**: Performance overhead due to additional layers

Virtualization with Containers



- **Containers**
  - A container is a **lightweight** and portable software package that encapsulates an application code and all its dependencies.
    - Dependencies include everything needed to run the code such as the files, configuration information, libraries, database, and operating system, etc. that matches the deployment environment.
- **Why containers?**
  - A VM is too big (2-4 GB for the smallest OS, compute cost in increment of 1 CPU/processor) that take time to create and transfer.
  - A container has most of the advantages of virtualization, but the image is much smaller (~5 MB), fast and efficient as opposed to heavy weight VM, gives increased portability.
- **Container runtime engine**
  - runs and manages containers, providing an interface between the host operating system and the container.
    - Therefore, a container instance must be **compatible** with the **hosting operating system**.
    - Several vendors of container runtime: Docker, Containerd, CRI-o

Virtualization with Containers

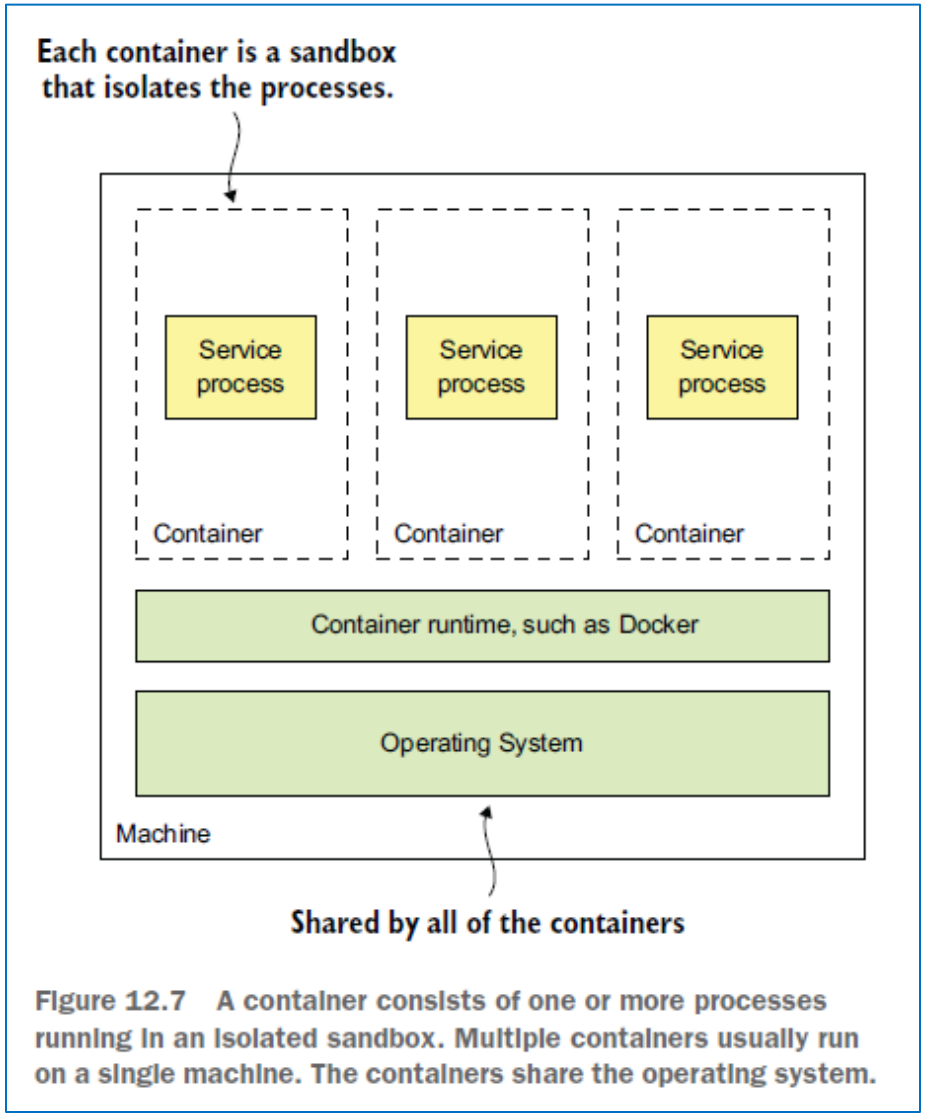
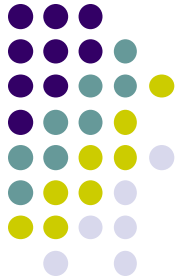


Figure 12.7 A container consists of one or more processes running in an isolated sandbox. Multiple containers usually run on a single machine. The containers share the operating system.

Containerization



- **Containerization**
  - Packaging "software code" and "all its dependencies" into an "image" so that it can run consistently on any infrastructure and share the image through a central server "registry"
- **Why containerization?**
  - Lightweight, little resource, fast load and transfer, better isolation among applications
    - A container provides a consistent runtime environment for applications regardless of the underlying infrastructure, operating system, or cloud platform.
    - You can create a container to run the LAMP stack (Linux, Apache, MySQL, and PHP).
- **Some platforms for containerization and container sharing**
  - **Docker** for creating containers
  - **Kubernetes** for container orchestration
  - Apache Mesos for cluster manager to share and manage containers

Containerization Process

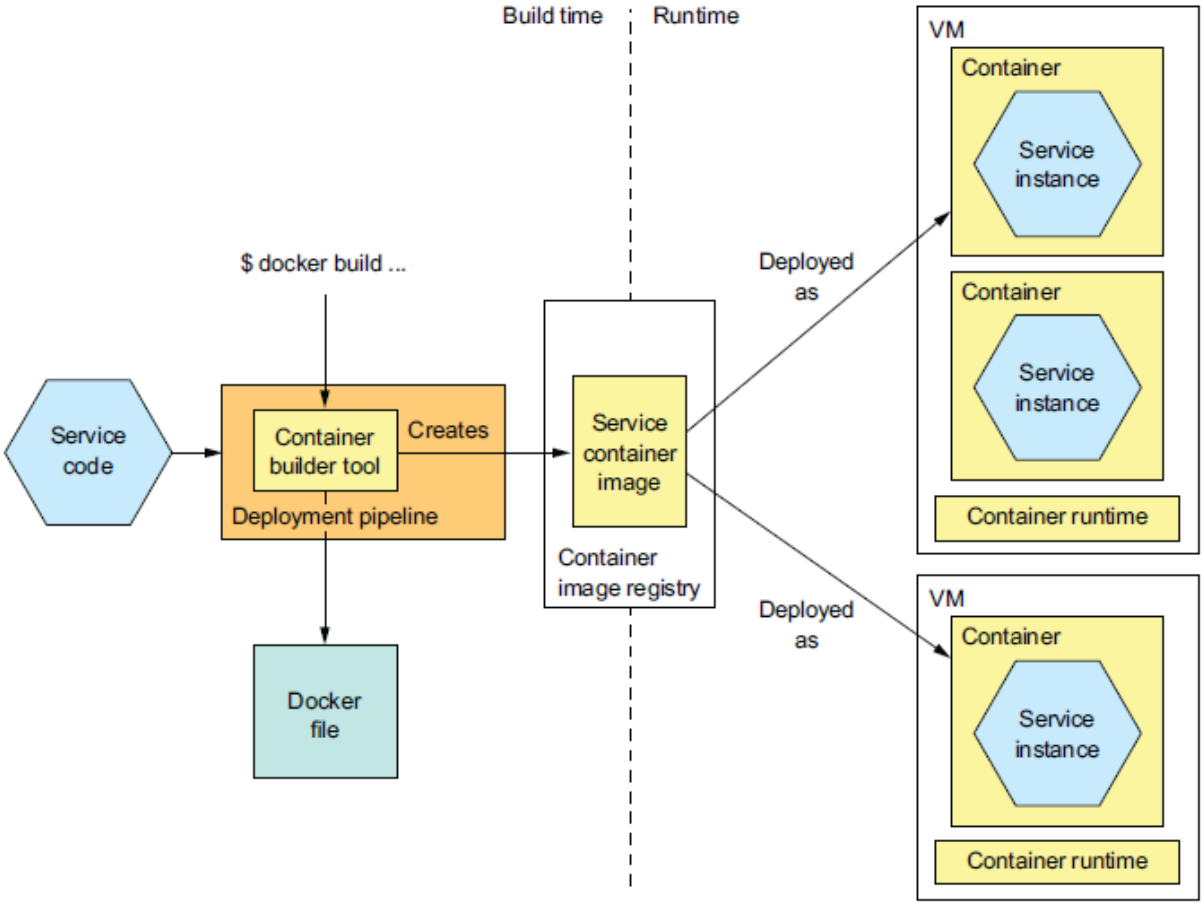
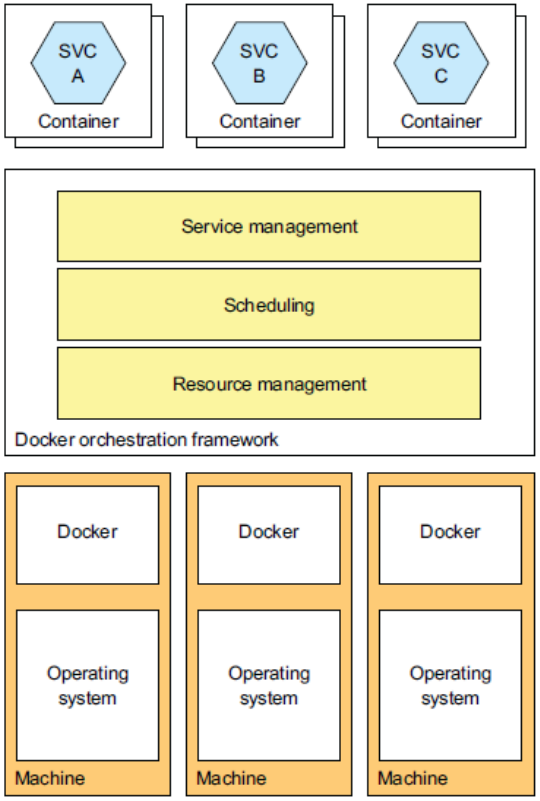


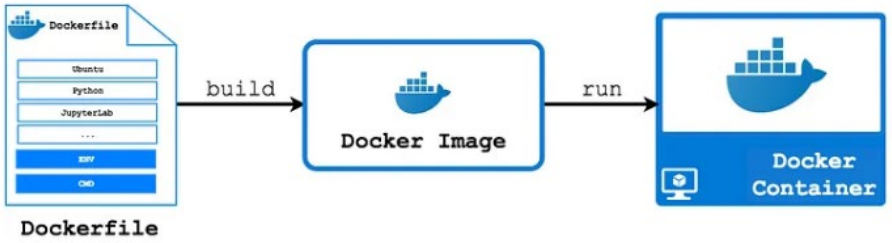
Figure 12.8 A service is packaged as a container image, which is stored in a registry. At runtime the service consists of multiple containers instantiated from that image. Containers typically run on virtual machines. A single VM will usually run multiple containers.

Docker



- **Docker**
  - A platform for containerization and running applications using container.
- **Docker vocabularies**
  - A container is created from a Docker image that contains the actual application.
    - An image may consist of one or more containers.
  - An image is a read-only template with instructions for creating container built from a Dockerfile and registered to a Docker hub.
    - A Docker file is a script that defines the application's environment.
    - Types of images: base (parent) images for operating systems (Ubuntu, BusyBox), child images (created and inherited from parent image), official images (pre-built and available on Docker hub), user images (custom images)
  - Docker hub is a registry of Docker images in a repository.
    - So, images can be pulled from the Docker hub.
  - Docker orchestration is the management of multiple containers running on a cluster of computers.
    - Deployment, scaling, networking, and load balancing using tools such as Docker swarm, Kubernetes, Apache Mesos
  - Volume is the mechanism for persisting data generated and used by Docker containers.

Creating a Dockerfile



- **Create a Dockerfile** (to run a Python program using container)
  - Create a Python program "my_classifier.py" and "requirements.txt".
  - Create the "Dockerfile" (this is the actual file name) in the same directory.

```
# Use an official Python runtime as the base image
FROM python:3.9-slim

# Set the working directory in the container
WORKDIR /myapp

# Copy the requirements file into the container
COPY requirements.txt .

# Install the Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy the rest of the application code into the container
COPY . .

# Specify the command to run your Python script
CMD ["python", "classifier.py"]
```

COPY . . will copy everything in the current code directory (excluded in .dockerignore) into the /myapp directory set by WORKDIR inside the container.

If external files are needed, you can copy them into the container using the COPY command in the Dockerfile.

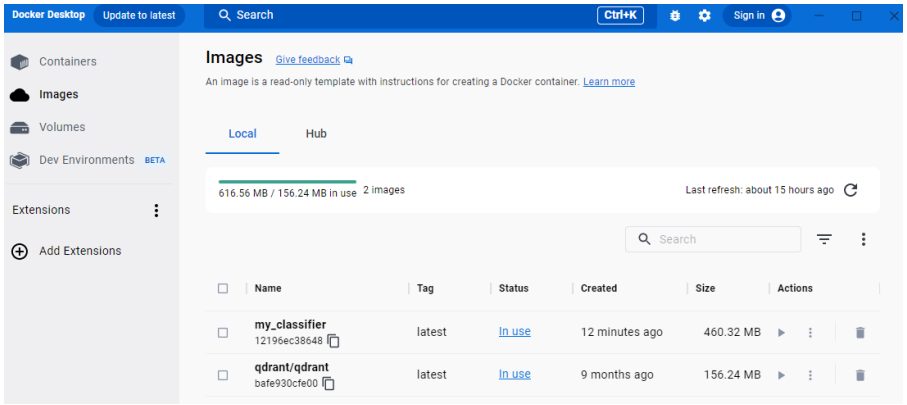
Building Image and Running Container

- **Building the Docker image**
  - "docker build –t my_classifier ."
    - "-t" is to tag (or label) the image name and "." refers to the current directory
    - To view the image: "docker image ls" (or you can use Docker desktop GUI)
- **Running the container**
  - "docker run --rm my_classifier"
    - --rm is to automatically remove the container after it finishes (instead of being stopped)
    - "docker rmi my_classifier" will remove the image. "-f" force removal

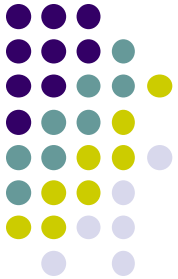
Docker command

```
PS C:\Users\taery\CSU Fullerton Dropbox\Christopher Ryu\Projects\Test> docker images
REPOSITORY      TAG       IMAGE ID       CREATED          SIZE
my_classifier   latest    12196ec38648   11 minutes ago   460MB
qdrant/qdrant   latest    bafe930cfe00   8 months ago     156MB
```

Docker desktop

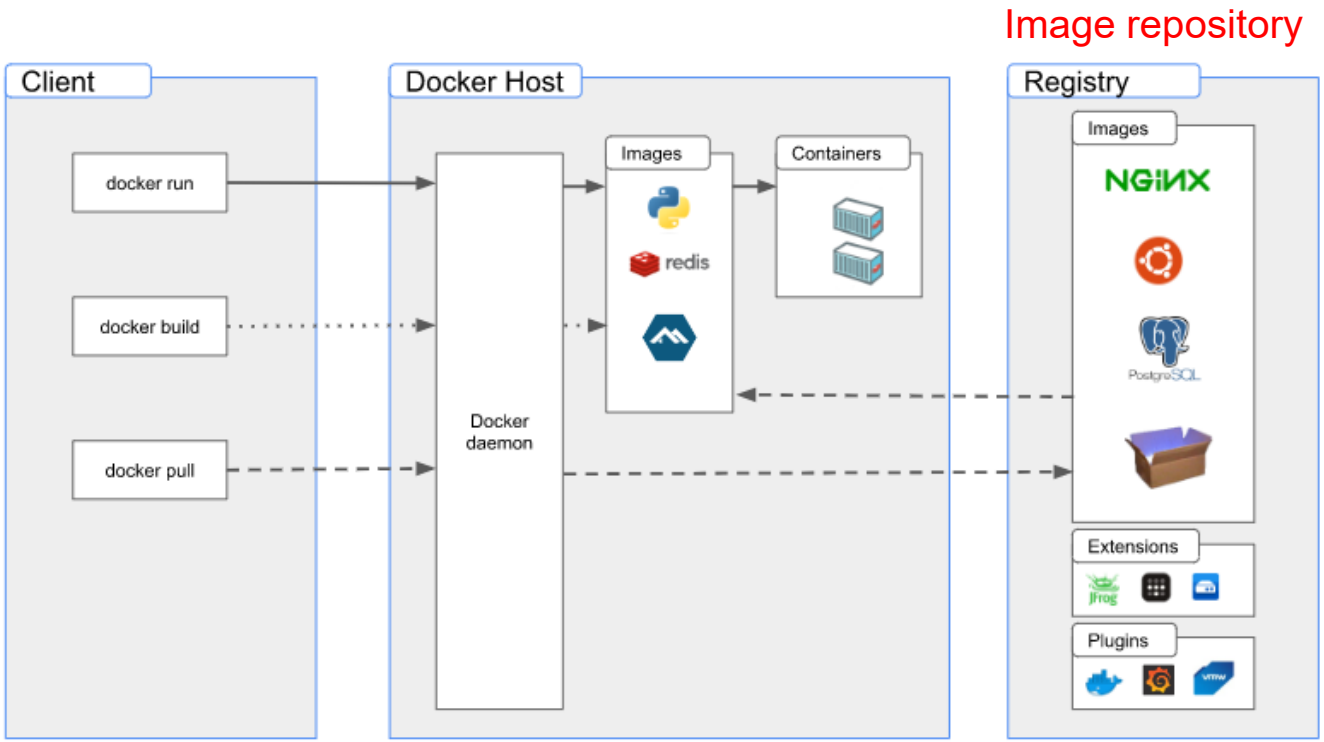


Publishing and Pulling Images



- **Publishing (pushing) images**
  - To publish images to Docker Hub, create a Docker Hub account and login
  - To push your image to Docker Hub, it must be tagged with your username and the image name.
    - <username>/<image>:<tag>, e.g., “john/my_classifier” (becomes repository name)
    - “docker tag my_classifier john/my_classifier:latest” (“latest” is the default)
  - “docker push john/my_classifier:latest”
- **Pulling (downloading) images**
  - “docker pull john/my_classifier:latest”
    - “docker pull” retrieves a Docker image from a registry to your local machine.
- For more information about Docker commands and examples:
  - https://www.docker.com/
  - https://docs.docker.com/
  - https://www.docker.com/101-tutorial/

Docker Architecture



- Docker client is the command line or GUI tool that allows the users to interact with the Docker daemon.
- Docker daemon is the background service running on the host that manages building, running, and distributing Docker containers.
- Docker host is the physical or VM on which the Docker runtime engine is running.
- Registry is a service that stores and distributes Docker images.

Kubernetes

Kubernetes cluster consists of a master node and worker nodes.

Control plane (master node) manages the worker nodes and pods.

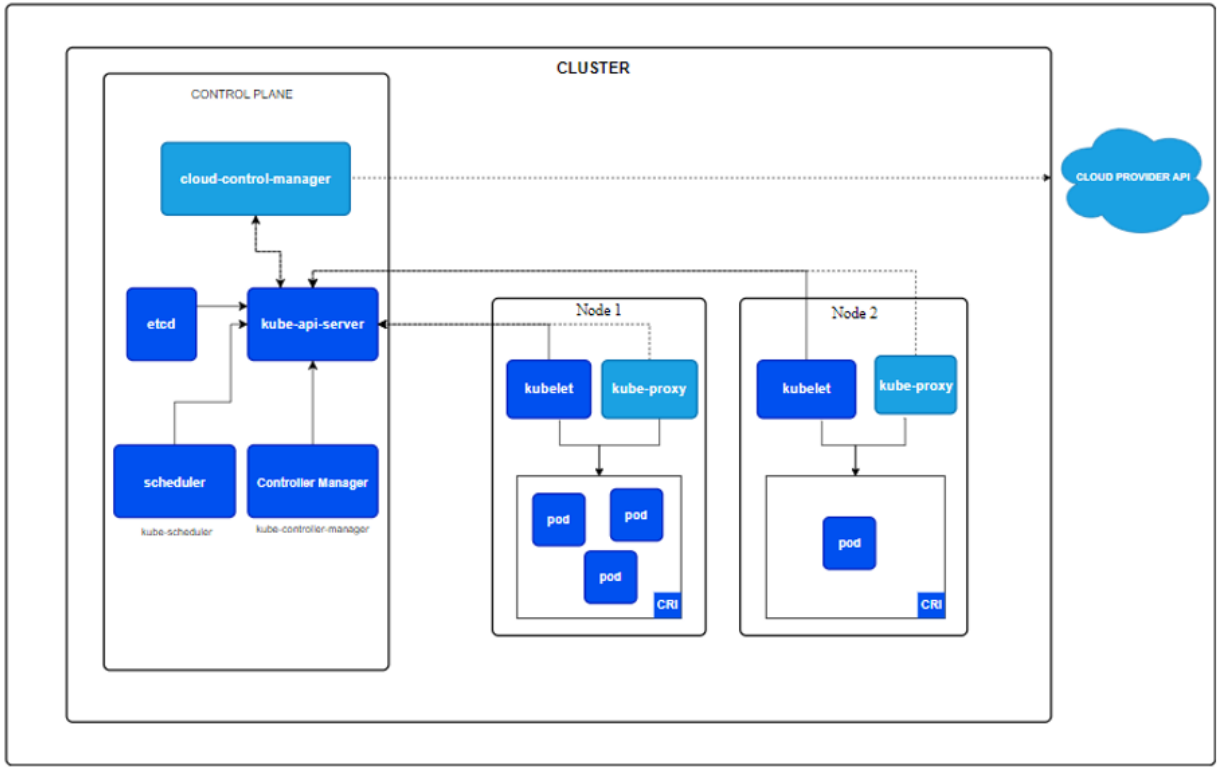
Node is the physical or virtual machine.

Pod is the smallest deployable unit (one or more containers), handling the application workload.

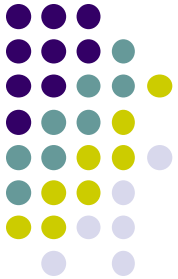
Kubelet is an agent that ensures containers run in a pod with the container runtime (e.g., Docker) via CRI (container runtime interface).

Kube-proxy manages network traffic on each node, enabling communication between pods, load balancing, service discovery, traffic routing.

Kubernetes cluster

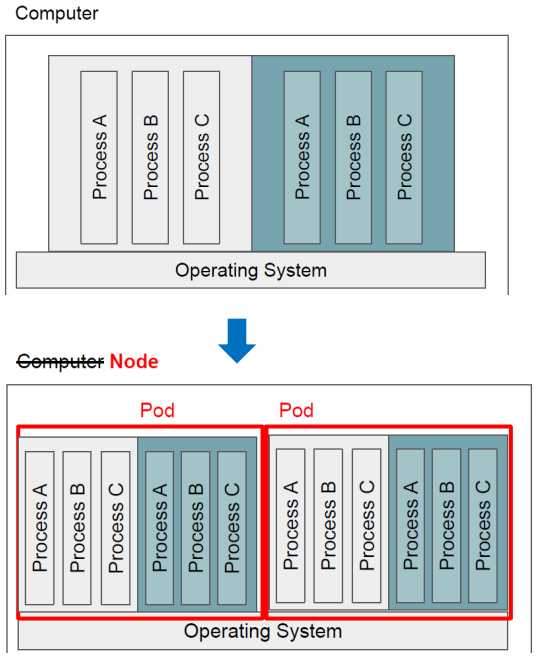


Workloads in the Kubernetes Cluster

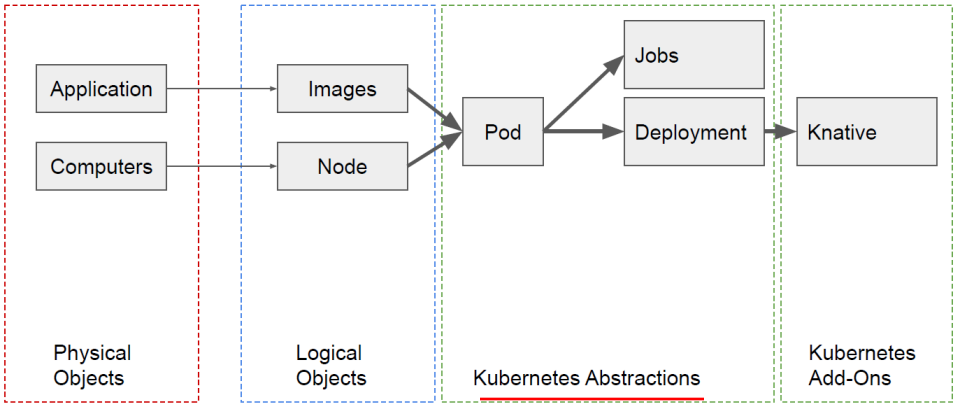


- **Workloads**
  - Applications running on Kubernetes
- **Types of workloads**
  - **Pod** is the smallest compute object, representing a *single instance* of a running process.
  - **Job** is a task (or controller) that ensures Pods run to completion.
    - Typically used for batch processing. Multiple Pods can run in parallel in a job.
  - **Deployment** is a way to manage a collection of Pods.
    - Modern deployment practice such as Infrastructure as Code (**IaC**), DevOps, CI/CD
  - Statefulset is a resource that manages the deployment and scaling of stateful application.
    - Unlike Deployment commonly used for stateless applications
  - Others such as DaemonSet, ReplicaSet, CronJob, etc.
- **Volume**
  - Persistent storage for containers, useful for stateful applications

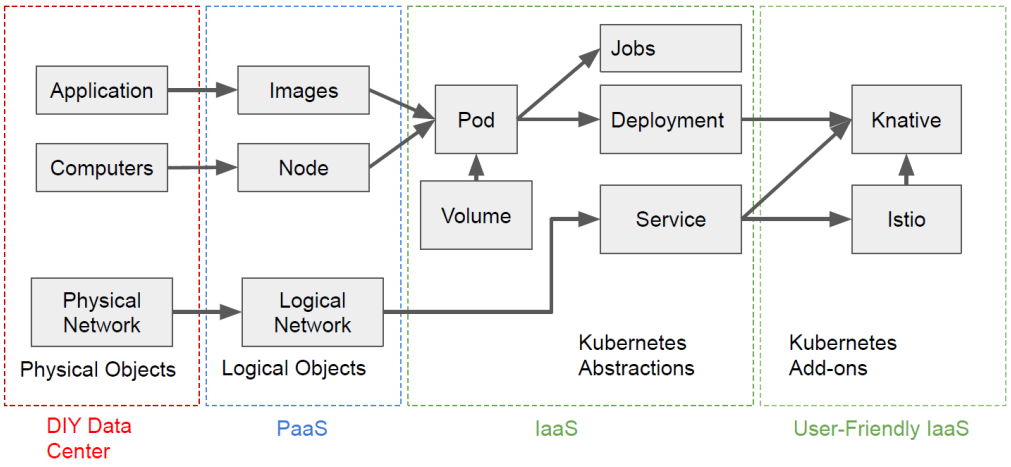
Kubernetes – Pod Execution

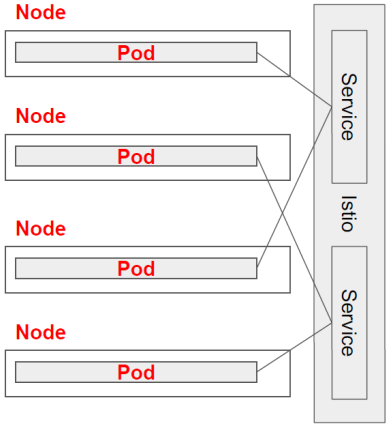


- **Pod**
  - A group of related containers that share an IP address and port to receive requests, communicate with each other using Inter-Process Communication (IPC) such as semaphore or shared memory.
  - Fate sharing: Processes in a pod (e.g., Web server, local SQL instance) live/die together as they are allocated and deallocated together.
  - Managed by controller
- **Why Pod** (instead of container)?
  - Scalability and fault tolerance as multiple Pods can become replicas and execute on different nodes dynamically.
  - Multi-tenancy as multiple pods can run on a single node and provide a high-level of isolation, scalability, efficient use of clustered resources, and security.



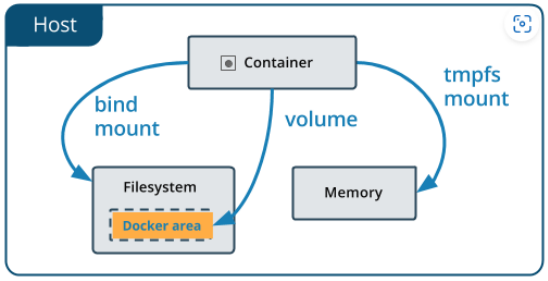
Kubernetes – Networking Model

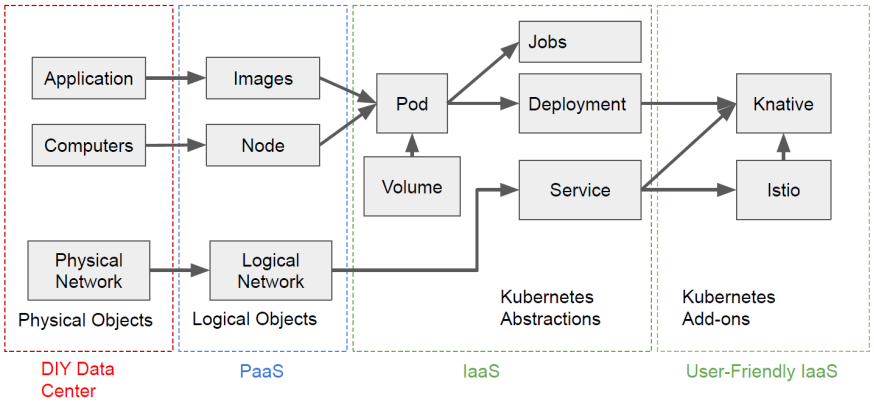




- **Networking model for Pods**
  - All containers in a Pod are in the same network (as if local).
  - All pods are in a flat address space (same subdomain).
    - but nodes are physical servers, they don't have to be in the same network address space in the data center.
  - Pods are dynamically allocated, replicated, deallocated, and can move around for many services. **Two problems:**
    - **How do Pods maintain the local data and states?** Use **Volume** that is a directory that can store data
    - **How do Pods find each other?** Use **service mesh** (**Istio** open-source project started by Google and IBM)

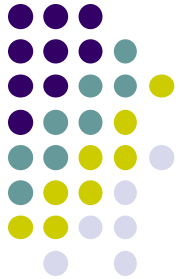
Kubernetes – Persistence





- **Types of Volume**
  - emptyDir Volume is created (like local disk) when a Pod is created and deleted when the Pod is removed.
  - hostPath Volume mounts a directory from the host node into a container in a Pod and used to share data between the container and the host.
  - configMap Volume stores configuration data (key-value pairs).
  - secret Volume stores sensitive information (password, or keys for data)
- **S3 bucket storage**
  - NRP Nautilus uses Ceph S3 protocol (open-source object storage over a network)
    - NOT AWS S3 bucket but compatible to AWS S3 bucket

Specifying Required Resources



```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
  - name: my-container
    image: ubuntu
    resources:
      requests:
        memory: "4500Mi"
        cpu: "4000m"
        nvidia.com/gpu: "1"
      limits:
        memory: "4750Mi"
        cpu: "4500m"
        nvidia.com/gpu: "1"
    volumeMounts:
    - name: workingdirectory
      mountPath: /working
  volumes:
  - name: workingdirectory
    emptyDir: {}
```

Some of the key elements of YAML for Docker containers in Kubernetes

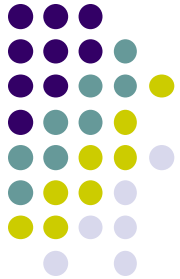
apiVersion, kind, metadata, spec, resources, env (environment variables), volumes and volumeMounts, livenesProbe and readinessProbe, replicas (in a Deployment), imagePullPolicy, service, Affinity and Toleration, Labels and Selectors, Autoscaling

apiVersion: Kubernetes API version
For a Pod: v1
for a Deployment: apps/v1
kind: Pod, Deployment, Service
metadata: about the object including name
spec: defines the desired state of the object such as container name, Docker image name
resources: defines the resource requests and limits, and memory usage for efficient management of resources.
Volumes and volumeMounts: defines storage that containers can use.

Use an **editing tool** like VS Code to create or edit a YAML file instead of a text editor as it is case-sensitive and requires indentation, specific syntax.

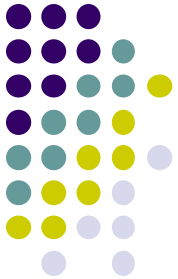
- **YAML** is a human-readable data serialization format (text) commonly used for configuration files and data serialization (alternative to JSON)

Infrastructure as a Code (IaC)



- **Challenges of deployment on Cloud**
  - Deployment on cloud is complex because building, configuring the necessary infrastructure components, and properly provisioning the required resources is difficult and time-consuming (especially by DevOps process requiring CI/CD practice).
- Infrastructure as Code
  - The process of automatically managing and provisioning compute resources and infrastructure through machine-readable definition files, enabling the best practices in DevOps instead of manually configuring them.
  - **Benefits**: Consistency, speed, version control, portability (multiple cloud platforms)
  - **Types of IaC**
    - Declarative defines the desired state of the infrastructure in a configuration file. The IaC tool determines the actions needed to achieve that desired state and executes them automatically.
    - Imperative defines specific commands (scripts) to specify how the infrastructure should be provisioned and configured (similar to traditional configuration management tool).
  - Some declarative IaC tools
    - Terraform (templates in both YAML and JSON, commonly used for configuration files), AWS CloudFormation, Azure Resource Manager, Google Cloud Deployment Manager.
  - Some imperative IaC tools
    - Ansible, Chef

Setting Up Kubernetes Environment



- **Set up a development environment on your computer**
  - Create a working directory (or folder) for development, say “test”.
  - Create a directory named “.kube” in the working directory.
  - Download “kubectl” command line interface
    - Or GUI wrapper or VS code plugin
- **Login to https://nrp.ai with your CSUF credential.**
  - Upon login to the system, your account will be “guest”. One of the Nautilus admin should promote “guest” to “user” by adding to the CSUF namespace “csuf-titans”.
  - Download a configuration file from the Nautilus page and save it under “.kube” directory.
    - The configuration file contains a token with your authentication and authorization information.
    - Refer to: https://nrp.ai/documentation/
- **Verify if everything is ready.**
  - Type “kubectl get namespace”. You should be able to see a list of namespaces.

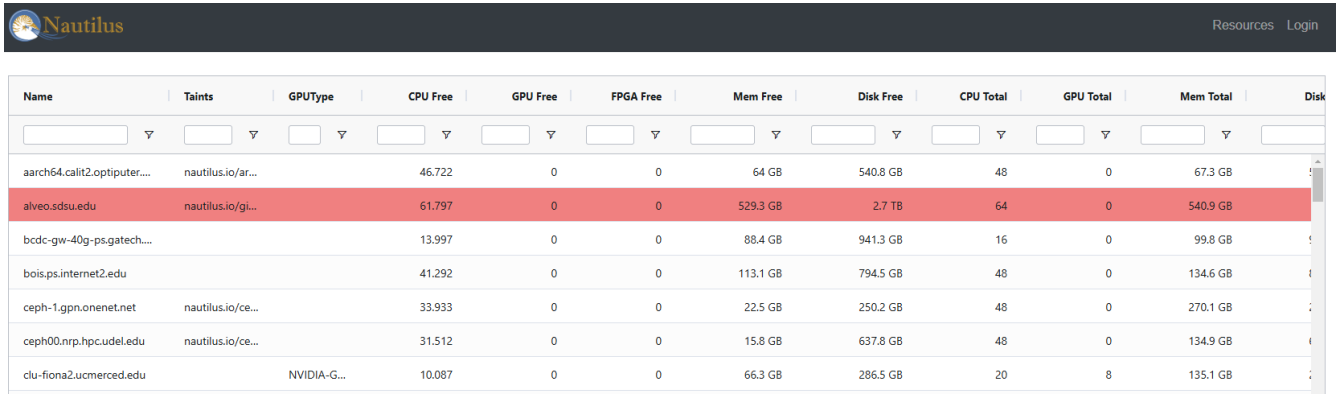
Deploying Applications using K8s Commands

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: t3-pod
  labels:
    app: t3-app
spec:
  containers:
  - name: t3-container
    image: ubuntu:latest
    # Keeps the container running for one hour
    command: ["sleep", "3600"]
```

1. **Creating and launching a Pod**
   - Create a YAML file that specifies the resources, e.g., "my-pod.yaml".
     - The available resources can be found at the Nautilus resource page.
     - You can request at most 1 GPU at the Nautilus.
   - Create, launch, and verify the Pod
     - Create a Pod, "kubectl apply –f my-pod.yaml" or "kubectl create".
     - Verify the Pod creation, "kubectl get pods" or "kubectl describe pod my-pod".
     - When a Pod is created, K8s will attempt to run the container defined in it. If there is no error, the Pod should start running. Otherwise, the Pod will fail.
2. **Deploying the program with the Pod**
   - Copy the program files from local machine to the Pod.
     - "kubectl **cp** test.py my-pod:." or "kubectl **cp** test.py my-pod:./t1.py"
   - Login to the Pod, "kubectl **exec** –it t3-pod -- /bin/bash".
   - Install the necessary packages inside the Pod
     - Install packages using pip command and run the program
   - Exit from the Pod, "exit", Linux command.
   - Make sure you remove the resources, "kubectl delete pod my-pod"!

Available Compute Resources

- **Find the GPU resources in the Nautilus resource page.**



Name	Taints	GPUType	CPU Free	GPU Free	FPGA Free	Mem Free	Disk Free	CPU Total	GPU Total	Mem Total	Disk
aarch64.calit2.optiputer....	nautilus.io/ar...		46.722	0	0	64 GB	540.8 GB	48	0	67.3 GB	
alveo.sdsu.edu	nautilus.io/gi...		61.797	0	0	529.3 GB	2.7 TB	64	0	540.9 GB	
bcdc-gw-40g-ps.gatech....			13.997	0	0	88.4 GB	941.3 GB	16	0	99.8 GB	
bois.ps.internet2.edu			41.292	0	0	113.1 GB	794.5 GB	48	0	134.6 GB	
ceph-1.gpn.onenet.net	nautilus.io/ce...		33.933	0	0	22.5 GB	250.2 GB	48	0	270.1 GB	
ceph00.nrp.hpc.udel.edu	nautilus.io/ce...		31.512	0	0	15.8 GB	637.8 GB	48	0	134.9 GB	
clu-fiona2.ucmerced.edu		NVIDIA-G...	10.087	0	0	66.3 GB	286.5 GB	20	8	135.1 GB	

- **Create a YAML file to specify the GPU resource.**

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: t3-pod
spec:
  containers:
  - name: t3-container
    image: my_classifier-image:latest
    resources:
      limits:
        nvidia.com/gpu: 1 # Request 1 GPU (NVIDIA-specific resource)
    command: ["python", "my_classifier.py"]
```

Pod name should be unique within the same namespace and resource type (kind).

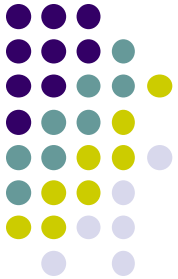
This command is needed only if your Dockerfile does not have CMD [“pyhon”, “my_classifier.py”]

Deploying Containerized Applications

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: t5-docker-pod
spec:
  containers:
    - name: t5-docker-container
      # replace with your username and Docker image name
      image: <dockerhub-username>/<image-name>:<tag>
      ports:
        - containerPort: 80 # optional
```

1. **Create and build a container image**
   - Install the Docker and Docker desktop.
   - Create a Dockerfile and build the container image with the Dockerfile using "docker build" command.
     - The Docker image should include all the program files and dependencies independently run your application.
   - Run the Docker image locally for testing using "docker run" command.
2. **Push the container image to a Registry**
   - Docker Hub or other private registry
3. **Deploy the container image**
   - Create a YAML file that specifies the resources (like the example).
   - Create and launch a Pod that runs your applications.
     - No need to copy files from local machine as the container has all the necessary files and packages

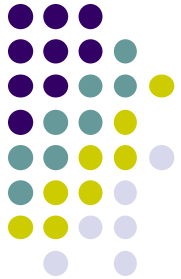
Multiple Containers within a Pod



- **Pod**
  - A basic deployable unit as a single instance of a running process
  - Multiple containers share:
    - Network namespace, storage volume, and lifecycle
- **Why multiple containers?**
  - Sidecar pattern where an auxiliary container does logging, monitoring, or proxy alongside the main container.
  - Adapter pattern where a secondary container transforms or process data for the main
  - Ambassador pattern where a secondary container acts as a proxy to facilitate communication with external services

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: multi-container-pod
spec:
  containers:
    - name: app-container
      image: nginx
      ports:
        - containerPort: 80
      volumeMounts:
        - name: shared-logs
          mountPath: /var/log/nginx

    - name: sidecar-container
      image: busybox
      command: ["/bin/sh", "-c", "tail -f /var/log/nginx/access.log"]
      volumeMounts:
        - name: shared-logs
          mountPath: /var/log/nginx

  volumes:
    - name: shared-logs
      emptyDir: {}
```

Deploying Jobs



- **Job**
  - A controller (workload or resource type) that ensures a task (that may need one or more Pods) run to completion
    - For batch processing or data migration
  - Automatic Pod restart on failure
  - Parallel execution by multiple Pods for distributed processing
- **Example: Multiple Pods**
  - At any given time, 2 Pods are running concurrently.
  - Each Pod has one instance of the worker container executing the job.
  - Once a Pod completes, a new one is created until all 5 are completed.

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: parallel-job
spec:
  completions: 5
  parallelism: 2
  template:
    spec:
      containers:
        - name: worker
          image: busybox
          command: ["/bin/sh", "-c", "echo Processing job; sleep 5"]
      restartPolicy: Never
```

Time	Pod 1	Pod 2	Pod 3	Pod 4	Pod 5
Start	Running	Running	-	-	-
After 5s	Finished	Running	Running	-	-
After 10s	Finished	Finished	Running	Running	-
After 15s	Finished	Finished	Finished	Running	Running
After 20s	Finished	Finished	Finished	Finished	Finished

Additional Information and Support

- **The tutorial page on the CS department page**
  - https://assessment.ecs.fullerton.edu/static/nautilus_tutorial.html
- Kubernetes: https://kubernetes.io/docs/reference/kubectl/quick-reference/
- JupyterHub and Notebook: https://jupyter.org
- Docker tool and getting started for container: https://docs.docker.com/
- Nautilus documentation and getting started
- The **matrix**: the Nautilus support group

1. Nautilus Documentation: https://ucsd-prp.gitlab.io/

2. Join the matrix: https://ucsd-prp.gitlab.io/userdocs/start/contact/